1*. Introduction. This program converts an `AWEB` file to an ADA file. It was written—originally for conversion to PASCAL—by D. E. Knuth in September, 1981; a somewhat similar SAIL program had been developed in March, 1979. Since this program describes itself, a bootstrapping process involving hand-translation had to be used to get started. The conversion to ADA has been done by U. Schweigert in May 1988.

For large `AWEB` files one should have a large memory, since `ATANGLE` keeps all the ADA text in memory (in an abbreviated form). This program uses only ADA-features as described in "The Programming Language ADA Reference Manual" and is thus conform with ANSI/MIL-STD-1815A-1983.

The system-dependent portions of `ATANGLE` can be identified by looking at the entries for 'system dependencies' in the index below.

The "banner line" defined here should be changed whenever `ATANGLE` is modified.

define *banner* ≡ `"This␣is␣ATANGLE,␣Version␣1.1␣(NRaD␣Version)"`

2. The program consists of several packages that are declared right now; each of these packages and either the specification and the body of the packages are send to a seperate file. The main program itself is declared later.

3*. The first package we declare is called *int_number_io* and enables us to output integer numbers. Further we declare a package *data_structures*, which will contain all the global constants, types, variables, and some procedures used throughout the program; it will (nearly) contain code of the sections "The character set," "Data structures," and "Tokens."

```
output to file data_structures.ads
with TEXT_IO;
with TEXT_IO;
with int_number_io;
package data_structures is
  ⟨ Types and objects visible from data_structures 10 ⟩
  ⟨ Specification of procedures visible from data_structures 47 ⟩
end data_structures;
output to file data_structures.adb
with input_output;
use input_output;
package body data_structures is
  ⟨ Procedures in data_structures 48 ⟩
begin
  ⟨ Set initial values 15 ⟩
end data_structures;
```

4*. The following package, called *input_output*, is responsible for the input/output features neaded by ATANGLE, and for the error handling. It will contain code of sections "Input and output" and "Reporting errors to the user."

output to file `input_output.ads`

```
with TEXT_IO;
with data_structures;
use data_structures;
package input_output is
  ⟨ Specification of procedures visible from input_output 23 ⟩
end input_output;
```

output to file `input_output.adb`

```
with TEXT_IO, int_number_io;
package body input_output is
  ⟨ Procedures in input_output 24 ⟩
end input_output;
```

5*. Additionally we have a package *hashing*, which contains the code of sections "Searching for identifiers" and "Searching for module names."

output to file `hashing.ads`

```
with data_structures;
use data_structures;
package hashing is
  ⟨ Specification of procedures visible from hashing 50 ⟩
end hashing;
```

output to file `hashing.adb`

```
with TEXT_IO;
with input_output;
use input_output;
package body hashing is
  ⟨ Procedures in hashing 51 ⟩
end hashing;
```

6*. The first of the two major packages will manage the output process of ATANGLE and contains code of sections "Stacks for output," "Producing the output," and "The big output switch."

output to file `output_phase.ads`

```
with data_structures;
use data_structures;
package output_phase is
  procedure phase_ii;
end output_phase;
```

output to file `output_phase.adb`

```
with TEXT_IO, int_number_io;
with input_output;
use input_output;
package body output_phase is
  ⟨ Procedures in output_phase 81 ⟩
end output_phase;
```

7*. The last package will handle `ATANGLE`'s input process and contains code of sections "Introduction to the input phase," "Inputting the next token," "Scanning a macro definition," and "Scanning a module."

```
output to file input_phase.ads
with data_structures;
use data_structures;
with hashing;
use hashing;
package input_phase is
  procedure phase_i;
end input_phase;
output to file input_phase.adb
with TEXT_IO, int_number_io;
with input_output;
use input_output;
package body input_phase is
  ⟨ Procedures in input_phase 119 ⟩
end input_phase;
```

8. Some of this code, delimited by **stat** and **tats**, is optionally included if statistics about `ATANGLE`'s memory usage are desired. Other material is optionally included when a somewhat optimal code is wanted; such is delimited by **optim** and **mitpo**.

```
define stat ≡ @{   { change this to 'stat ≡ ' when gathering usage statistics }
define tats ≡ @}   { change this to 'tats ≡ ' when gathering usage statistics }
format stat ≡ declare
format tats ≡ end
define optim ≡ @{   { change this to 'optim ≡ ' when optimal code is desired }
define mitpo ≡ @}   { change this to 'mitpo ≡ ' when optimal code is desired }
format optim ≡ declare
format mitpo ≡ end
```

9. Here are some macros for common programming idioms.

```
define incr(#) ≡ # ← # + 1   { increase a variable by unity }
define decr(#) ≡ # ← # − 1   { decrease a variable by unity }
define repeat ≡ loop
define until(#) ≡ exit when #;
      end loop
format repeat ≡ loop
format until ≡ end
```

10. The following parameters are set big enough to handle TEX, so they should be sufficient for most applications of ATANGLE.

⟨ Types and objects visible from *data_structures* 10 ⟩ ≡
buf_size : **constant** ← 100; { maximum length of input line }
max_bytes : **constant** ← 45_000; { 1/*ww* times the number of bytes in identifiers, strings, and module names; must be less than 65_536 }
max_toks : **constant** ← 50_000;
{ 1/*zz* times the number of bytes in compressed ADA code; must be less than 65_536 }
max_names : **constant** ← 4_000;
{ number of identifiers, strings, module names; must be less than 10_240 }
max_texts : **constant** ← 2_000; { number of replacement texts, must be less than 10_240 }
hash_size : **constant** ← 353; { should be prime }
longest_name : **constant** ← 400; { module names shouldn't be longer than this }
line_length : **constant** ← 72; { lines of ADA output have at most this many characters }
out_buf_size : **constant** ← 144; { length of output buffer, should be twice *line_length* }
stack_size : **constant** ← 50; { number of simultaneous levels of macro expansion }
max_id_length : **constant** ← *line_length*;
{ long identifiers are chopped to this length, which must not exceed *line_length* }
unambig_length : **constant** ← *max_id_length*; { identifiers must be unique if chopped to this length }

See also sections 11, 12, 13, 14, 16, 22, 25, 27, 30, 35, 38, 39, 40, 42, 44, 45, 49, 62, 63, 70, 72, 76, 77, 79, 83, 91, 92, 97, 107, 109, 116, 118, 131, 136, 148, 156, and 165.

This code is used in section 3*.

11. A global variable called *history* will contain one of four values at the end of every run: *spotless* means that no unusual messages were printed; *harmless_message* means that a message of possible interest was printed but no serious errors were detected; *error_message* means that at least one error was found; *fatal_message* means that the program terminated abnormally. The value of *history* does not influence the behavior of the program; it is simply computed for the convenience of systems that might want to use such information.

define *mark_harmless* ≡
if *history* = *spotless* **then**
history ← *harmless_message*;
end if
define *mark_error* ≡ *history* ← *error_message*
define *mark_fatal* ≡ *history* ← *fatal_message*

⟨ Types and objects visible from *data_structures* 10 ⟩ +≡
type *history_type* **is** (*spotless*, *harmless_message*, *error_message*, *fatal_message*);
history : *history_type* ← *spotless*; { how bad was this run? }

12. The character set. One of the main goals in the design of AWEB has been to make it readily portable between a wide variety of computers. Yet AWEB by its very nature must use a greater variety of characters than most computer programs deal with, and character encoding is one of the areas in which existing machines differ most widely from each other.

To resolve this problem, all input to AWEAVE and ATANGLE is converted to an internal seven-bit code that is essentially standard ASCII, the "American Standard Code for Information Interchange." The conversion is done immediately when each character is read in. Conversely, characters are converted from ASCII to the user's external representation just before they are output.

Such an internal code is relevant to users of AWEB only because it is the code used for preprocessed constants like @'A'. If you are writing a program in AWEB that makes use of such one-character constants, you should convert your input to ASCII form, like AWEAVE and ATANGLE do. Otherwise AWEB's internal coding scheme does not affect you.

Here is a table of the standard visible ASCII codes:

	0	*1*	*2*	*3*	*4*	*5*	*6*	*7*
040_8	␣	!	"	#	$	%	&	'
050_8	(	)	*	+	,	-	.	/
060_8	0	1	2	3	4	5	6	7
070_8	8	9	:	;	<	=	>	?
100_8	@	A	B	C	D	E	F	G
110_8	H	I	J	K	L	M	N	O
120_8	P	Q	R	S	T	U	V	W
130_8	X	Y	Z	[	\	]	^	_
140_8	`	a	b	c	d	e	f	g
150_8	h	i	j	k	l	m	n	o
160_8	p	q	r	s	t	u	v	w
170_8	x	y	z	{	|	}	~	

(Actually, of course, code 40_8 is an invisible blank space.) Code 136_8 was once an upward arrow (↑), and code 137_8 was once a left arrow (←), in olden times when the first draft of ASCII code was prepared; but AWEB works with today's standard ASCII in which those codes represent circumflex and underline as shown.

⟨ Types and objects visible from *data_structures* 10 ⟩ +≡
subtype *ASCII_code* **is** INTEGER **range** 0 . . 127; { seven-bit numbers, a subrange of the integers }

13. We shall use the name *text_char* to stand for the data type of the characters in the input and output files. We shall also assume that *text_char* consists of the elements *text_char*'FIRST through *text_char*'LAST, inclusive. The following definition should be adjusted if necessary.

⟨ Types and objects visible from *data_structures* 10 ⟩ +≡
subtype *text_char* **is** CHARACTER; { the data type of characters in text files }
subtype *text_file* **is** TEXT_IO.FILE_TYPE; { file type consisting of *text_char* }

14. The AWEAVE and ATANGLE processors convert between ASCII code and the user's external character set by means of arrays *xord* and *xchr* that are analogous to ADA's POS and VAL attributes.

⟨ Types and objects visible from *data_structures* 10 ⟩ +≡
xord : **array** (*text_char*) **of** *ASCII_code*; { specifies conversion of input characters }
xchr : **array** (*ASCII_code*) **of** *text_char*; { specifies conversion of output characters }

15. If we assume that every system using `AWEB` is able to read and write the visible characters of standard ASCII (although not necessarily using the ASCII codes to represent them), the following assignment statements initialize most of the *xchr* array properly, without needing any system-dependent changes. For example, the statement `xchr(8#101#):='A';` that appears in the present `AWEB` file might be encoded in, say, EBCDIC code on the external medium on which it resides, but `ATANGLE` will convert from this external code to ASCII and back again. Therefore the assignment statement `XCHR(8#101#):='A';` will appear in the corresponding ADA file, and ADA will compile this statement so that $xchr(101_8)$ receives the character `A` in the external (*text_char*) code. Note that it would be quite incorrect to say `xchr(8#101#):=@'A';`, because [`A`] is a constant of type `INTEGER`, not `CHARACTER`, and because we have [`A`] = 65 regardless of the external character set.

⟨ Set initial values 15 ⟩ ≡

$xchr(40_8) \leftarrow$ ´␣´; $xchr(41_8) \leftarrow$ ´!´; $xchr(42_8) \leftarrow$ ´"´; $xchr(43_8) \leftarrow$ ´#´; $xchr(44_8) \leftarrow$ ´$´;
$xchr(45_8) \leftarrow$ ´%´; $xchr(46_8) \leftarrow$ ´&´; $xchr(47_8) \leftarrow$ ´´´;
$xchr(50_8) \leftarrow$ ´(´; $xchr(51_8) \leftarrow$ ´)´; $xchr(52_8) \leftarrow$ ´*´; $xchr(53_8) \leftarrow$ ´+´; $xchr(54_8) \leftarrow$ ´,´;
$xchr(55_8) \leftarrow$ ´-´; $xchr(56_8) \leftarrow$ ´.´; $xchr(57_8) \leftarrow$ ´/´;
$xchr(60_8) \leftarrow$ ´0´; $xchr(61_8) \leftarrow$ ´1´; $xchr(62_8) \leftarrow$ ´2´; $xchr(63_8) \leftarrow$ ´3´; $xchr(64_8) \leftarrow$ ´4´;
$xchr(65_8) \leftarrow$ ´5´; $xchr(66_8) \leftarrow$ ´6´; $xchr(67_8) \leftarrow$ ´7´;
$xchr(70_8) \leftarrow$ ´8´; $xchr(71_8) \leftarrow$ ´9´; $xchr(72_8) \leftarrow$ ´:´; $xchr(73_8) \leftarrow$ ´;´; $xchr(74_8) \leftarrow$ ´<´;
$xchr(75_8) \leftarrow$ ´=´; $xchr(76_8) \leftarrow$ ´>´; $xchr(77_8) \leftarrow$ ´?´;
$xchr(100_8) \leftarrow$ ´@´; $xchr(101_8) \leftarrow$ ´A´; $xchr(102_8) \leftarrow$ ´B´; $xchr(103_8) \leftarrow$ ´C´; $xchr(104_8) \leftarrow$ ´D´;
$xchr(105_8) \leftarrow$ ´E´; $xchr(106_8) \leftarrow$ ´F´; $xchr(107_8) \leftarrow$ ´G´;
$xchr(110_8) \leftarrow$ ´H´; $xchr(111_8) \leftarrow$ ´I´; $xchr(112_8) \leftarrow$ ´J´; $xchr(113_8) \leftarrow$ ´K´; $xchr(114_8) \leftarrow$ ´L´;
$xchr(115_8) \leftarrow$ ´M´; $xchr(116_8) \leftarrow$ ´N´; $xchr(117_8) \leftarrow$ ´O´;
$xchr(120_8) \leftarrow$ ´P´; $xchr(121_8) \leftarrow$ ´Q´; $xchr(122_8) \leftarrow$ ´R´; $xchr(123_8) \leftarrow$ ´S´; $xchr(124_8) \leftarrow$ ´T´;
$xchr(125_8) \leftarrow$ ´U´; $xchr(126_8) \leftarrow$ ´V´; $xchr(127_8) \leftarrow$ ´W´;
$xchr(130_8) \leftarrow$ ´X´; $xchr(131_8) \leftarrow$ ´Y´; $xchr(132_8) \leftarrow$ ´Z´; $xchr(133_8) \leftarrow$ ´[´; $xchr(134_8) \leftarrow$ ´\´;
$xchr(135_8) \leftarrow$ ´]´; $xchr(136_8) \leftarrow$ ´^´; $xchr(137_8) \leftarrow$ ´_´;
$xchr(140_8) \leftarrow$ ´`´; $xchr(141_8) \leftarrow$ ´a´; $xchr(142_8) \leftarrow$ ´b´; $xchr(143_8) \leftarrow$ ´c´; $xchr(144_8) \leftarrow$ ´d´;
$xchr(145_8) \leftarrow$ ´e´; $xchr(146_8) \leftarrow$ ´f´; $xchr(147_8) \leftarrow$ ´g´;
$xchr(150_8) \leftarrow$ ´h´; $xchr(151_8) \leftarrow$ ´i´; $xchr(152_8) \leftarrow$ ´j´; $xchr(153_8) \leftarrow$ ´k´; $xchr(154_8) \leftarrow$ ´l´;
$xchr(155_8) \leftarrow$ ´m´; $xchr(156_8) \leftarrow$ ´n´; $xchr(157_8) \leftarrow$ ´o´;
$xchr(160_8) \leftarrow$ ´p´; $xchr(161_8) \leftarrow$ ´q´; $xchr(162_8) \leftarrow$ ´r´; $xchr(163_8) \leftarrow$ ´s´; $xchr(164_8) \leftarrow$ ´t´;
$xchr(165_8) \leftarrow$ ´u´; $xchr(166_8) \leftarrow$ ´v´; $xchr(167_8) \leftarrow$ ´w´;
$xchr(170_8) \leftarrow$ ´x´; $xchr(171_8) \leftarrow$ ´y´; $xchr(172_8) \leftarrow$ ´z´; $xchr(173_8) \leftarrow$ ´{´; $xchr(174_8) \leftarrow$ ´|´;
$xchr(175_8) \leftarrow$ ´}´; $xchr(176_8) \leftarrow$ ´~´;
$xchr(0) \leftarrow$ ´␣´; $xchr(177_8) \leftarrow$ ´␣´; { these ASCII codes are not used }

See also sections 17, 18, 41, 43, 46, 71, and 142.

This code is used in section 3*.

16. Some of the ASCII codes below 40_8 have been given symbolic names in AWEAVE and ATANGLE because they are used with a special meaning.

⟨ Types and objects visible from *data_structures* 10 ⟩ +≡
and_sign : **constant** *ASCII_code* ← 4_8; { equivalent to 'and' }
not_sign : **constant** *ASCII_code* ← 5_8; { equivalent to 'not' }
set_element_sign : **constant** *ASCII_code* ← 6_8; { equivalent to 'in' }
tab_mark : **constant** *ASCII_code* ← 11_8; { ASCII code used as tab-skip }
line_feed : **constant** *ASCII_code* ← 12_8; { ASCII code thrown away at end of line }
form_feed : **constant** *ASCII_code* ← 14_8; { ASCII code used at end of page }
carriage_return : **constant** *ASCII_code* ← 15_8; { ASCII code used at end of line }
right_arrow : **constant** *ASCII_code* ← 21_8; { equivalent to '=>' }
box_sign : **constant** *ASCII_code* ← 22_8; { equivalent to '<>' }
double_star : **constant** *ASCII_code* ← 23_8; { equivalent to '**' }
left_label_bracket : **constant** *ASCII_code* ← 24_8; { equivalent to '<<' }
right_label_bracket : **constant** *ASCII_code* ← 25_8; { equivalent to '>>' }
left_arrow : **constant** *ASCII_code* ← 30_8; { equivalent to ':=' }
not_equal : **constant** *ASCII_code* ← 32_8; { equivalent to '/=' }
less_or_equal : **constant** *ASCII_code* ← 34_8; { equivalent to '<=' }
greater_or_equal : **constant** *ASCII_code* ← 35_8; { equivalent to '>=' }
equivalence_sign : **constant** *ASCII_code* ← 36_8; { equivalent to '==' }
or_sign : **constant** *ASCII_code* ← 37_8; { equivalent to 'or' }

17. Here now is the system-dependent part of the character set. If AWEB is being implemented on a garden-variety ADA for which only standard ASCII codes will appear in the input and output files, you don't need to make any changes here.

Changes to the present module will make AWEB more friendly on computers that have an extended character set, so that one can type things like ≠ instead of /=. If you have an extended set of characters that are easily incorporated into text files, you can assign codes arbitrarily here, giving an *xchr* equivalent to whatever characters the users of AWEB are allowed to have in their input files, provided that unsuitable characters do not correspond to special codes like *carriage_return* that are listed above.

(The present file ATANGLE.AWEB does not contain any of the non-ASCII characters, because it is intended to be used with all implementations of AWEB.)

⟨ Set initial values 15 ⟩ +≡
xchr(1 .. 37_8) ← (**others** ⇒ '␣');

18. The following system-independent code makes the *xord* array contain a suitable inverse to the information in *xchr*.

⟨ Set initial values 15 ⟩ +≡
xord(*text_char*) ← (**others** ⇒ 40_8);
for $i \in 1 \mathinner{..} 176_8$ **loop**
  xord(*xchr*(i)) ← i;
end loop;

19. Input and output. The input conventions of this program are identical to those of AWEAVE. Therefore people who need to make modifications to both systems should be able to do so without too many headaches.

20. Terminal output is done by writing on file STANDARD_OUTPUT, which is assumed to consist of characters of type *text_char*:

```
define new_ln ≡ TEXT_IO.NEW_LINE
define print(#) ≡ TEXT_IO.PUT(#)   { 'print' means write on the terminal }
define print_ln(#) ≡ print(#); new_ln   { 'print' and then start new line }
define print_nl(#) ≡ new_ln; print(#)   { print information starting on a new line }
define print_int(#) ≡ int_number_io.PUT(#)   { print out an integer value }
```

21. The *update_terminal* procedure is called when we want to make sure that everything we have output to the terminal so far has actually left the computer's internal buffers and been sent. (In the ADA-system used to develop the initial program this is not required.)

```
define update_terminal ≡ null   { empty the terminal output buffer }
```

22. The main input comes from *web_file*; this input may be overridden by changes in *change_file*. (If *change_file* is empty, there are no changes.)

```
⟨ Types and objects visible from data_structures 10 ⟩ +≡
  web_file : text_file;   { primary input }
  change_file : text_file;   { updates }
```

23.

```
⟨ Specification of procedures visible from input_output 23 ⟩ ≡
  procedure open_a_file(f : in out text_file; mode_of_file : TEXT_IO.FILE_MODE; default_name : STRING;
        success : out BOOLEAN);
  function create_ada_file(ada_file_name : STRING)return BOOLEAN;
```

See also sections 28 and 31.

This code is used in section 4*.

24. The following code opens a file and returns TRUE if succesfully. If the file is already open the procedure has no effect; this gives the ability to open the files by means of the change file, without changing the AWEB-file of ATANGLE.

```
⟨ Procedures in input_output 24 ⟩ ≡
  procedure open_a_file(f : in out text_file; mode_of_file : TEXT_IO.FILE_MODE; default_name : STRING;
        success : out BOOLEAN) is
  begin
    if TEXT_IO.IS_OPEN(f) then
      success ← TRUE;
    else
      TEXT_IO.OPEN(FILE ⇒ f, MODE ⇒ mode_of_file, NAME ⇒ default_name); success ← TRUE;
    end if;
  exception
    when TEXT_IO.STATUS_ERROR | TEXT_IO.NAME_ERROR | TEXT_IO.USE_ERROR ⇒
      success ← FALSE; return;
    end open_a_file;
```

See also sections 26, 29, and 32.

This code is used in section 4*.

25. The main output goes to *ada_file*.

```
⟨ Types and objects visible from data_structures 10 ⟩ +≡
  ada_file : text_file;
```

26. The following code creates *ada_file*.

```
⟨ Procedures in input_output 24 ⟩ +≡
  function create_ada_file(ada_file_name : STRING)return BOOLEAN is
  begin
    if TEXT_IO.IS_OPEN(ada_file) then
      if TEXT_IO.NAME(ada_file) = ada_file_name then
        return TRUE;
      else
        return FALSE;
      end if;
    else
      TEXT_IO.CREATE(FILE ⇒ ada_file, MODE ⇒ TEXT_IO.OUT_FILE, NAME ⇒ ada_file_name);
      return TRUE;
    end if;
  exception
    when TEXT_IO.STATUS_ERROR | TEXT_IO.NAME_ERROR | TEXT_IO.USE_ERROR ⇒
      return FALSE;
    end create_ada_file;
```

27. Input goes into an array called *buffer*.

```
⟨ Types and objects visible from data_structures 10 ⟩ +≡
  type buffer_type is array (INTEGER range <>) of ASCII_code;
  subtype buffer_index is INTEGER range 0 .. buf_size;
  buffer : buffer_type(buffer_index);
```

28. The *input_ln* procedure brings the next line of input from the specified file into the *buffer* array and returns the value TRUE, unless the file has already been entirely read, in which case it returns FALSE. The conventions of TEX are followed; i.e., *ASCII_code* numbers representing the next line of the file are input into *buffer*(0), *buffer*(1), ..., *buffer*(*limit* − 1); trailing blanks are ignored; and the global variable *limit* is set to the length of the line. The value of *limit* must be strictly less than *buf_size*.

We assume that none of the *ASCII_code* values of *buffer*(j) for $0 \le j < limit$ is equal to 0, 177_8, *line_feed*, *form_feed*, or *carriage_return*.

```
⟨ Specification of procedures visible from input_output 23 ⟩ +≡
  function input_ln(f : text_file)return BOOLEAN;
```

29. ⟨ Procedures in *input_output* 24 ⟩ +≡

```
function input_ln(f : text_file)return BOOLEAN is    { inputs a line or returns FALSE }
  final_limit : buffer_index ← 0;   { limit without trailing blanks }
  c : text_char;
begin
  limit ← 0;
  if ¬TEXT_IO.IS_OPEN(f) or else TEXT_IO.END_OF_FILE(f) then
    return FALSE;
  else
    while ¬TEXT_IO.END_OF_LINE(f) loop
      TEXT_IO.GET(f, c); buffer(limit) ← xord(c); incr(limit);
      if buffer(limit − 1) ≠ [␣] then
        final_limit ← limit;
      end if;
      if limit = buf_size then
        TEXT_IO.SKIP_LINE(f); decr(limit);   { keep buffer(buf_size) empty }
        print_nl("!␣Input␣line␣too␣long"); loc ← 0; error; return TRUE;
      end if;
    end loop;
    TEXT_IO.SKIP_LINE(f); limit ← final_limit; return TRUE;
  end if;
end input_ln;
```

30. Reporting errors to the user. The ATANGLE processor operates in two phases: first it inputs the source file and stores a compressed representation of the program, then it produces the ADA output from the compressed representation.

The global variable *phase_one* tells whether we are in Phase I or not.

⟨ Types and objects visible from *data_structures* 10 ⟩ +≡
  phase_one : BOOLEAN; { TRUE in Phase I, FALSE in Phase II }

31. During the first phase, syntax errors are reported to the user by saying

'*err_print*("!␣Error␣message")',

followed by '*jump_out*' if no recovery from the error is provided. This will print the error message followed by an indication of where the error was spotted in the source file. Note that no period follows the error message, since the error routine will automatically supply a period.

Errors that are noticed during the second phase are reported to the user in the same fashion, but the error message will be followed by an indication of where the error was spotted in the output file.

The actual error indications are provided by a procedure called *error*.

define *err_print*(#) ≡ *print_nl*(#); *error*

⟨ Specification of procedures visible from *input_output* 23 ⟩ +≡
  procedure *error*;

32. ⟨ Procedures in *input_output* 24 ⟩ +≡
  procedure *error* **is** { prints '.' and location of error message }
    l : *buffer_index*; { index into *buffer* }
  begin
    if *phase_one* **then**
      ⟨ Print error location based on input buffer 33 ⟩;
    else
      ⟨ Print error location based on output buffer 34 ⟩;
    end if;
    update_terminal; *mark_error*;
  end *error*;

33. The error locations during Phase I can be indicated by using the global variables *loc*, *line*, and *changing*, which tell respectively the first unlooked-at position in *buffer*, the current line number, and whether or not the current line is from *change_file* or *web_file*. This routine should be modified on systems whose standard text editor has special line-numbering conventions.

```
⟨ Print error location based on input buffer 33 ⟩ ≡
  if changing then
    print(".␣(change␣file␣");
  else
    print(".␣(");
  end if;
  print("l."); print_int(line, 1); print_ln(")");
  if loc ≥ limit then
    l ← limit;
  else
    l ← loc;
  end if;
  for k ∈ 1 .. l loop
    if buffer(k − 1) = tab_mark then
      print("␣");
    else
      print(xchr(buffer(k − 1)));  { print the characters already read }
    end if;
  end loop;
  new_ln;
  for k ∈ 1 .. l loop
    print("␣");
  end loop;  { space out the next line }
  for k ∈ l + 1 .. limit loop
    print(xchr(buffer(k − 1)));
  end loop;  { print the part not yet read }
  print("␣")  { this space separates the message from future asterisks }
```

This code is used in section 32.

34. The position of errors detected during the second phase can be indicated by outputting the partially-filled output buffer, which contains *out_ptr* entries. The current output file name is determined by *file_name* and *fn_length*, whereas the line number comes from *f_line*.

```
⟨ Print error location based on output buffer 34 ⟩ ≡
  print(".(" & file_name(1 .. fn_length) & "␣l."); print_int(f_line, 1); print_ln(")");
  for j ∈ 1 .. out_ptr loop
    print(xchr(out_buf(j − 1)));  { print current partial line }
  end loop;
  print("...␣")  { indicate that this information is partial }
```

This code is used in section 32.

35. The *jump_out* macro just raises the *end_of_ATANGLE* exception. It is used when no recovery from a particular error has been provided.

```
define jump_out ≡ raise end_of_ATANGLE
define fatal_error(#) ≡ print_nl(#); error; mark_fatal; jump_out
⟨ Types and objects visible from data_structures 10 ⟩ +≡
  end_of_ATANGLE : exception;
```

36. Sometimes the program's behavior is far different from what it should be, and ATANGLE prints an error message that is really for the ATANGLE maintenance person, not the user. In such cases the program says *confusion*("indication␣of␣where␣we␣are").

define *confusion*(#) ≡ *fatal_error*("!␣This␣can´t␣happen␣(" & # & ")")

37. An overflow stop occurs if ATANGLE's tables aren't large enough.

define *overflow*(#) ≡ *fatal_error*("!␣Sorry,␣" & # & "␣capacity␣exceeded")

38. Data structures. Most of the user's ADA code is packed into seven- or eight-bit integers in two large arrays called *byte_mem* and *tok_mem*. The *byte_mem* array holds the names of identifiers, strings, and modules; the *tok_mem* array holds the replacement texts for macros and modules. Allocation is sequential, since things are deleted only during Phase II, and only in a last-in-first-out manner.

Auxiliary arrays *byte_start* and *tok_start* are used as directories to *byte_mem* and *tok_mem*, and the *link*, *ilk*, *equiv*, and *text_link* arrays give further information about names. These auxiliary arrays consist of sixteen-bit items.

⟨ Types and objects visible from *data_structures* 10 ⟩ +≡
  subtype *eight_bits* **is** INTEGER **range** 0 . . 255; { unsigned one-byte quantity }
  subtype *sixteen_bits* **is** INTEGER **range** 0 . . 65_535; { unsigned two-byte quantity }

39. ATANGLE has been designed to avoid the need for indices that are more than sixteen bits wide, so that it can be used on most computers. But there are programs that need more than 65536 tokens, and some programs even need more than 65536 bytes; TEX is one of these. To get around this problem, a slight complication has been added to the data structures: *byte_mem* and *tok_mem* are two-dimensional arrays, whose first index is either 0 or 1. (For generality, the first index is actually allowed to run between 0 and $ww - 1$ in *byte_mem*, or between 0 and $zz - 1$ in *tok_mem*, where ww and zz are set to 2 and 3; the program will work for any positive values of ww and zz, and it can be simplified in obvious ways if $ww = 1$ or $zz = 1$.)

⟨ Types and objects visible from *data_structures* 10 ⟩ +≡
  ww : **constant** INTEGER ← 2;
  zz : **constant** INTEGER ← 3;

  subtype *w_range* **is** INTEGER **range** 0 . . *ww* − 1;
  subtype *z_range* **is** INTEGER **range** 0 . . *zz* − 1;
  subtype *byte_index* **is** INTEGER **range** 0 . . *max_bytes*;
  subtype *token_index* **is** INTEGER **range** 0 . . *max_toks*;
  subtype *name_index* **is** INTEGER **range** 0 . . *max_names*;
  subtype *text_index* **is** INTEGER **range** 0 . . *max_texts*;

  type *byte_memory* **is array** (*w_range*, *byte_index*) **of** *ASCII_code*;
  optim
    pragma PACK(*byte_memory*);
  mitpo

  type *token_memory* **is array** (*z_range*, *token_index*) **of** *eight_bits*;
  optim
    pragma PACK(*token_memory*);
  mitpo

  type *name_info* **is array** (*name_index*) **of** *sixteen_bits*;
  type *token_info* **is array** (*text_index*) **of** *sixteen_bits*;

  byte_mem : *byte_memory*; { characters of names }
  tok_mem : *token_memory*; { tokens }
  byte_start : *name_info*; { directory into *byte_mem* }
  tok_start : *token_info*; { directory into *tok_mem* }
  link : *name_info*; { hash table or tree links }
  ilk : *name_info*; { type codes or tree links }
  equiv : *name_info*; { info corresponding to names }
  text_link : *token_info*; { relates replacement texts }

40. The names of identifiers are found by computing a hash address h and then looking at strings of bytes signified by $hash(h)$, $link(hash(h))$, $link(link(hash(h)))$, ..., until either finding the desired name or encountering a zero.

A '*name_pointer*' variable, which signifies a name, is an index into *byte_start*. The actual sequence of characters in the name pointed to by p appears in positions $byte_start(p)$ to $byte_start(p+ww)-1$, inclusive, in the segment of *byte_mem* whose first index is p **mod** ww. Thus, when $ww = 2$ the even-numbered name bytes appear in $byte_mem(0,*)$ and the odd-numbered ones appear in $byte_mem(1,*)$. The pointer 0 is used for undefined module names; we don't want to use it for the names of identifiers, since 0 stands for a null pointer in a linked list.

The total number of names in *byte_mem* is called *name_ptr* and the total number of bytes occupied in $byte_mem(w,*)$ is called $byte_ptr(w)$.

We usually have $byte_start(name_ptr + w) = byte_ptr((name_ptr + w)$ **mod** $ww)$ for $0 \le w < ww$, since these are the starting positions for the next ww names to be stored in *byte_mem*.

define $length(\#) \equiv byte_start(\# + ww) - byte_start(\#)$ { the length of a name }
define $null_array \equiv$ (**others** $\Rightarrow 0$)

⟨ Types and objects visible from *data_structures* 10 ⟩ +≡
subtype *name_pointer* **is** *name_index*; { identifies a name }
name_ptr : *name_pointer* ← 1; { first unused position in *byte_start* }
byte_ptr : **array** (*w_range*) **of** *byte_index* ← *null_array*; { first unused position in *byte_mem* }

41. ⟨ Set initial values 15 ⟩ +≡
$byte_start(0 \,..\, ww) \leftarrow null_array$; { this makes name 0 of length zero }

42. Replacement texts are stored in *tok_mem*, using similar conventions. A '*text_pointer*' variable is an index into *tok_start*, and the replacement text that corresponds to p runs from positions $tok_start(p)$ to $tok_start(p + zz) - 1$, inclusive, in the segment of *tok_mem* whose first index is p **mod** zz. Thus, when $zz = 2$ the even-numbered replacement texts appear in $tok_mem(0,*)$ and the odd-numbered ones appear in $tok_mem(1,*)$. Furthermore, $text_link(p)$ is used to connect pieces of text that have the same name, as we shall see later. The pointer 0 is used for undefined replacement texts.

The first position of $tok_mem(z,*)$ that is unoccupied by replacement text is called $tok_ptr(z)$, and the first unused location of *tok_start* is called *text_ptr*. We usually have the identity $tok_start(text_ptr + z) = tok_ptr((text_ptr + z)$ **mod** $zz)$, for $0 \le z < zz$, since these are the starting positions for the next zz replacement texts to be stored in *tok_mem*.

It is convenient to maintain a variable z that is equal to *text_ptr* **mod** zz, so that we always insert tokens into segment z of *tok_mem*.

⟨ Types and objects visible from *data_structures* 10 ⟩ +≡
subtype *text_pointer* **is** *text_index*; { identifies a replacement text }
text_ptr : *text_pointer* ← 1; { first unused position in *tok_start* }
tok_ptr : **array** (*z_range*) **of** *token_index* ← *null_array*;
{ first unused position in a given segment of *tok_mem* }
z : *z_range* ← 1 **mod** *zz*; { current segment of *tok_mem* }
stat
max_tok_ptr : **array** (*z_range*) **of** *token_index*; { largest values assumed by *tok_ptr* }
tats

43. ⟨ Set initial values 15 ⟩ +≡
$tok_start(0 \,..\, zz) \leftarrow null_array$; { this makes replacement text 0 of length zero }

44. Three types of identifiers are distinguished by their *ilk*:

normal identifiers will appear in the ADA program as ordinary identifiers since they have not been defined to be macros; the corresponding value in the *equiv* array for such identifiers is a link in a secondary hash table that is used to check whether any two of them agree in their first *unambig_length* characters after letters are changed to uppercase.

simple identifiers have been defined to be simple macros; their *equiv* value points to the corresponding replacement text.

parametric identifiers have been defined to be parametric macros; like simple identifiers, their *equiv* value points to the replacement text.

⟨ Types and objects visible from *data_structures* 10 ⟩ +≡
normal : **constant** *sixteen_bits* ← 0; { ordinary identifiers have *normal* ilk }
simple : **constant** *sixteen_bits* ← 1; { simple macros have *simple* ilk }
parametric : **constant** *sixteen_bits* ← 2; { parametric macros have *parametric* ilk }

45. The names of modules are stored in *byte_mem* together with the identifier names, but a hash table is not used for them because `ATANGLE` needs to be able to recognize a module name when given a prefix of that name. A conventional binary seach tree is used to retrieve module names, with fields called *llink* and *rlink* in place of *link* and *ilk*. The root of this tree is *rlink*(0). If p is a pointer to a module name, *equiv*(p) points to its replacement text, just as in simple and parametric macros, unless this replacement text has not yet been defined (in which case *equiv*(p) = 0).

⟨ Types and objects visible from *data_structures* 10 ⟩ +≡
llink : *name_info* **renames** *link*; { left link in binary search tree for module names }
rlink : *name_info* **renames** *ilk*; { right link in binary search tree for module names }

46. ⟨ Set initial values 15 ⟩ +≡
rlink(0) ← 0; { the binary search tree starts out with nothing in it }
equiv(0) ← 0; { the undefined module has no replacement text }

47. Here is a little procedure that prints the text of a given name.

⟨ Specification of procedures visible from *data_structures* 47 ⟩ ≡
procedure *print_id*(p : *name_pointer*);

See also section 73.

This code is used in section 3*.

48. ⟨ Procedures in *data_structures* 48 ⟩ ≡

```
procedure print_id(p : name_pointer) is    { print identifier or module name }
  w : w_range;  { segment of byte_mem }
begin
  if p ≥ name_ptr then
    print("IMPOSSIBLE");
  else
    w ← p mod ww;
    for k ∈ byte_start(p) .. byte_start(p + ww) − 1 loop
      print(xchr(byte_mem(w, k)));
    end loop;
  end if;
end print_id;
```

See also section 74.

This code is used in section 3*.

49. Searching for identifiers. The hash table described above is updated by the *id_lookup* procedure, which finds a given identifier and returns a pointer to its index in *byte_start*. If the identifier was not already present, it is inserted with a given *ilk* code; and an error message is printed if the identifier is being doubly defined.

Because of the way ATANGLE's scanning mechanism works, it is most convenient to let *id_lookup* search for an identifier that is present in the *buffer* array. Two other global variables specify its position in the buffer: the first character is *buffer*(*id_first*), and the last is *buffer*(*id_loc* − 1).

We have mentioned that *normal* identifiers belong to two hash tables, one for their true names as they appear in the AWEB file and the other when they have been reduced to their first *unambig_length* characters. The hash tables are kept by the method of simple chaining, where the heads of the individual lists appear in the *hash* and *chop_hash* arrays. If h is a hash code, the primary hash table list starts at *hash*(h) and proceeds through *link* pointers; the secondary hash table list starts at *chop_hash*(h) and proceeds through *equiv* pointers. Of course, the same identifier will probably have two different values of h.

The *id_lookup* procedure uses an auxiliary array called *chopped_id* to contain up to *unambig_length* characters of the current identifier, if it is necessary to compute the secondary hash code.

⟨ Types and objects visible from *data_structures* 10 ⟩ +≡
  id_first : *buffer_index*; { where the current identifier begins in the buffer }
  id_loc : *buffer_index*; { just after the current identifier in the buffer }

  subtype *hash_index* **is** INTEGER **range** 0 .. *hash_size*;
  subtype *chop_hash_index* **is** INTEGER **range** 0 .. *unambig_length*;
  hash, *chop_hash* : **array** (*hash_index*) **of** *sixteen_bits* ← *null_array*;
    { heads of hash lists—initially empty }
  chopped_id : **array** (*chop_hash_index*) **of** *ASCII_code*; { chopped identifier }

50. Here now is the main procedure for finding identifiers (and strings). The parameter t is set to *normal* except when the identifier is a macro name that is just being defined; in the latter case, t will be *simple* or *parametric*.

⟨ Specification of procedures visible from *hashing* 50 ⟩ ≡
  function *id_lookup*(t : *eight_bits*)**return** *name_pointer*;

See also sections 64 and 68.

This code is used in section 5*.

51. ⟨ Procedures in *hashing* 51 ⟩ ≡

```
function id_lookup(t : eight_bits)return name_pointer is   { finds current identifier }
  c : eight_bits;  { byte being chopped }
  i : buffer_index;  { index into buffer }
  h : hash_index;  { hash code }
  k : byte_index;  { index into byte_mem }
  w : w_range;  { segment of byte_mem }
  l : buffer_index ← id_loc − id_first;  { length of the given identifier }
  p, q : name_pointer;  { where the identifier is being sought }
  s : chop_hash_index;  { index into chopped_id }
begin
  ⟨ Compute the hash code h 52 ⟩;
  ⟨ Compute the name location p 53 ⟩;
  if (p = name_ptr) ∨ (t ≠ normal) then
    ⟨ Update the tables and check for possible errors 55 ⟩;
  end if;
  return p;
end id_lookup;
```

See also sections 65 and 69.

This code is used in section 5*.

52. A simple hash code is used: If the sequence of ASCII codes is $c_1 c_2 \ldots c_n$, its hash value will be

$$(2^{n-1}c_1 + 2^{n-2}c_2 + \cdots + c_n) \bmod hash_size.$$

⟨ Compute the hash code h 52 ⟩ ≡

```
h ← buffer(id_first); i ← id_first + 1;
while i < id_loc loop
  h ← (h + h + buffer(i)) mod hash_size; incr(i);
end loop
```

This code is used in section 51.

53. If the identifier is new, it will be placed in position $p = name_ptr$, otherwise p will point to its existing location.

⟨ Compute the name location p 53 ⟩ ≡

```
p ← hash(h);
while p ≠ 0 loop
  if length(p) = l then
    ⟨ Compare name p with current identifier, goto found if equal 54 ⟩;
  end if;
  p ← link(p);
end loop;
p ← name_ptr;  { the current identifier is new }
link(p) ← hash(h); hash(h) ← p;  { insert p at beginning of hash list }
⟪found⟫ null
```

This code is used in section 51.

54. ⟨ Compare name p with current identifier, **goto** *found* if equal 54 ⟩ ≡

```
i ← id_first; k ← byte_start(p); w ← p mod ww;
while (i < id_loc) ∧ (buffer(i) = byte_mem(w, k)) loop
  incr(i); incr(k);
end loop;
if i = id_loc then
  goto found;  { all characters agree }
end if
```

This code is used in section 53.

55. ⟨ Update the tables and check for possible errors 55 ⟩ ≡

```
if ((p ≠ name_ptr) ∧ (t ≠ normal) ∧ (ilk(p) = normal)) ∨ ((p = name_ptr) ∧ (t = normal)) then
  ⟨ Compute the secondary hash code h and put the first characters into the auxiliary array
      chopped_id 56 ⟩;
end if;
if p ≠ name_ptr then
  ⟨ Give double-definition error and change p to type t 57 ⟩;
else
  ⟨ Enter a new identifier into the table at position p 59 ⟩;
end if
```

This code is used in section 51.

56. The following routine, which is called into play when it is necessary to look at the secondary hash table, computes the same hash function as before (but on the chopped data), and places a zero after the chopped identifier in *chopped_id* to serve as a convenient sentinel.

⟨ Compute the secondary hash code h and put the first characters into the auxiliary array *chopped_id* 56 ⟩ ≡

```
i ← id_first; s ← 0; h ← 0;
while (i < id_loc) ∧ (s < unambig_length) loop
  if buffer(i) ≥ [a] then
    chopped_id(s) ← buffer(i) − 40₈;
  else
    chopped_id(s) ← buffer(i);
  end if;
  h ← (h + h + chopped_id(s)) mod hash_size; incr(s); incr(i);
end loop;
chopped_id(s) ← 0
```

This code is used in section 55.

57. If a macro has appeared before it was defined, ATANGLE will still work all right; after all, such behavior is typical of the replacement texts for modules, which act very much like macros. However, ATANGLE finds it simplest to make a blanket rule that macros should be defined before they are used. The following routine gives an error message and also fixes up any damage that may have been caused.

```
⟨ Give double-definition error and change p to type t 57 ⟩ ≡
    { now p ≠ name_ptr and t ≠ normal }
  if ilk(p) = normal then
    err_print("!␣This␣identifier␣has␣already␣appeared");
    ⟨ Remove p from secondary hash table 58 ⟩;
  else
    err_print("!␣This␣identifier␣was␣defined␣before");
  end if;
  ilk(p) ← t
```

This code is used in section 55.

58. When we have to remove a secondary hash entry, because a *normal* identifier is changing to another *ilk*, the hash code h and chopped identifier have already been computed.

```
⟨ Remove p from secondary hash table 58 ⟩ ≡
  q ← chop_hash(h);
  if q = p then
    chop_hash(h) ← equiv(p);
  else
    while equiv(q) ≠ p loop
      q ← equiv(q);
    end loop;
    equiv(q) ← equiv(p);
  end if
```

This code is used in section 57.

59. ⟨ Enter a new identifier into the table at position p 59 ⟩ ≡

```
  if t = normal then
    ⟨ Check for ambiguity and update secondary hash 60 ⟩;
  end if;
  w ← name_ptr mod ww; k ← byte_ptr(w);
  if k + l > max_bytes then
    overflow("byte␣memory");
  end if;
  if name_ptr > max_names − ww then
    overflow("name");
  end if;
  i ← id_first;  { get ready to move the identifier into byte_mem }
  while i < id_loc loop
    byte_mem(w, k) ← buffer(i); incr(k); incr(i);
  end loop;
  byte_ptr(w) ← k; byte_start(name_ptr + ww) ← k; incr(name_ptr); ilk(p) ← t
```

This code is used in section 55.

60. ⟨ Check for ambiguity and update secondary hash 60 ⟩ ≡

```
q ← chop_hash(h);
while q ≠ 0 loop
  ⟨ Check if q conflicts with p 61 ⟩;
  q ← equiv(q);
end loop;
equiv(p) ← chop_hash(h); chop_hash(h) ← p   { put p at front of secondary list }
```

This code is used in section 59.

61. ⟨ Check if q conflicts with p 61 ⟩ ≡

```
k ← byte_start(q); s ← 0; w ← q mod ww;
while (k < byte_start(q + ww)) ∧ (s < unambig_length) loop
  c ← byte_mem(w, k);
  if c ≥ [a] then
    c ← c − 40_8;   { convert to uppercase }
  end if;
  if chopped_id(s) ≠ c then
    goto not_found;
  end if;
  incr(s); incr(k);
end loop;
if (k = byte_start(q + ww)) ∧ (chopped_id(s) ≠ 0) then
  goto not_found;
end if;
print_nl("!␣Identifier␣conflict␣with␣");
for kl ∈ byte_start(q) .. byte_start(q + ww) − 1 loop
  print(xchr(byte_mem(w, kl)));
end loop;
error; q ← 0;   { only one conflict will be printed, since equiv(0) = 0 }
⟪not_found⟫ null
```

This code is used in section 60.

62. Searching for module names. The *mod_lookup* procedure finds the module name *mod_text*(1 . . *l*) in the search tree, after inserting it if necessary, and returns a pointer to where it was found.

```
⟨ Types and objects visible from data_structures 10 ⟩ +≡
  subtype long_name_index is INTEGER range 0 .. longest_name;
  mod_text : array (long_name_index) of ASCII_code;   { name being sought for }
```

63. According to the rules of AWEB, no module name should be a proper prefix of another, so a "clean" comparison should occur between any two names. The result of *mod_lookup* is 0 if this prefix condition is violated. An error message is printed when such violations are detected during phase two of AWEAVE.

```
⟨ Types and objects visible from data_structures 10 ⟩ +≡
  type compare is (less, equal, greater, prefix, extension);
```

64.

```
⟨ Specification of procedures visible from hashing 50 ⟩ +≡
  function mod_lookup(l : sixteen_bits)return name_pointer;
```

65.

```
⟨ Procedures in hashing 51 ⟩ +≡
  function mod_lookup(l : sixteen_bits)return name_pointer is    { finds module name }
    c : compare ← greater;   { comparison between two names }
    j : long_name_index;   { index into mod_text }
    k : byte_index;   { index into byte_mem }
    w : w_range;   { segment of byte_mem }
    p : name_pointer;   { current node of the search tree }
    q : name_pointer ← 0;   { father of node p }
  begin
    p ← rlink(0);   { rlink(0) is the root of the tree }
    while p ≠ 0 loop
      ⟨ Set c to the result of comparing the given name to name p 67 ⟩;
      q ← p;
      if c = less then
        p ← llink(q);
      elsif c = greater then
        p ← rlink(q);
      else
        goto found;
      end if;
    end loop;
    ⟨ Enter a new module name into the tree 66 ⟩;
  ⟨⟨found⟩⟩ if c ≠ equal then
      err_print("!␣Incompatible␣section␣names");
      p ← 0;
    end if;
    return p;
  end mod_lookup;
```

66. ⟨ Enter a new module name into the tree 66 ⟩ ≡

```
w ← name_ptr mod ww; k ← byte_ptr(w);
if k + l > max_bytes then
   overflow("byte␣memory");
end if;
if name_ptr > max_names − ww then
   overflow("name");
end if;
p ← name_ptr;
if c = less then
   llink(q) ← p;
else
   rlink(q) ← p;
end if;
llink(p) ← 0; rlink(p) ← 0; c ← equal; equiv(p) ← 0;
for kj ∈ 1 . . l loop
   byte_mem(w, k + kj − 1) ← mod_text(kj);
end loop;
byte_ptr(w) ← k + l; byte_start(name_ptr + ww) ← k + l; incr(name_ptr)
```

This code is used in section 65.

67. ⟨ Set c to the result of comparing the given name to name p 67 ⟩ ≡

```
k ← byte_start(p); w ← p mod ww; c ← equal; j ← 1;
while (k < byte_start(p + ww)) ∧ (j ≤ l) ∧ (mod_text(j) = byte_mem(w, k)) loop
   incr(k); incr(j);
end loop;
if k = byte_start(p + ww) then
   if j > l then
      c ← equal;
   else
      c ← extension;
   end if;
elsif j > l then
   c ← prefix;
elsif mod_text(j) < byte_mem(w, k) then
   c ← less;
else
   c ← greater;
end if
```

This code is used in sections 65 and 69.

68. The *prefix_lookup* procedure is supposed to find exactly one module name that has *mod_text*(1 . . l) as a prefix. Actually the algorithm silently accepts also the situation that some module name is a prefix of *mod_text*(1 . . l), because the user who painstakingly typed in more than necessary probably doesn't want to be told about the wasted effort.

⟨ Specification of procedures visible from *hashing* 50 ⟩ +≡

```
function prefix_lookup(l : sixteen_bits)return name_pointer;
```

69. ⟨ Procedures in *hashing* 51 ⟩ +≡

```
function prefix_lookup(l : sixteen_bits)return name_pointer is   { finds name extension }
  c : compare;  { comparison between two names }
  count : name_index ← 0;  { the number of hits }
  j : long_name_index;  { index into mod_text }
  k : byte_index;  { index into byte_mem }
  w : w_range;  { segment of byte_mem }
  p : name_pointer;  { current node of the search tree }
  q : name_pointer ← 0;  { another place to resume the search after one branch is done }
  r : name_pointer ← 0;  { extension found }
begin
  p ← rlink(0);  { begin search at root of tree }
  while p ≠ 0 loop
    ⟨ Set c to the result of comparing the given name to name p 67 ⟩;
    if c = less then
      p ← llink(p);
    elsif c = greater then
      p ← rlink(p);
    else
      r ← p; incr(count); q ← rlink(p); p ← llink(p);
    end if;
    if p = 0 then
      p ← q; q ← 0;
    end if;
  end loop;
  if count ≠ 1 then
    if count = 0 then
      err_print("!␣Name␣does␣not␣match");
    else
      err_print("!␣Ambiguous␣prefix");
    end if;
  end if;
  return r;  { the result will be 0 if there was no match }
end prefix_lookup;
```

70. Tokens. Replacement texts, which represent ADA code in a compressed format, appear in *tok_mem* as mentioned above. The codes in these texts are called 'tokens'; some tokens occupy two consecutive eight-bit byte positions, and the others take just one byte.

If $p > 0$ points to a replacement text, *tok_start*(p) is the *tok_mem* position of the first eight-bit code of that text. If *text_link*$(p) = 0$, this is the replacement text for a macro, otherwise it is the replacement text for a module. In the latter case *text_link*(p) is either equal to *module_flag*, which means that there is no further text for this module, or *text_link*(p) points to a continuation of this replacement text; such links are created when several modules have ADA texts with the same name, and they also tie together all the ADA texts of unnamed modules. The replacement text pointer for the first unnamed module appears in *text_link*(0), and the most recent such pointer is *last_unnamed*.

⟨ Types and objects visible from *data_structures* 10 ⟩ +≡
module_flag : **constant** *sixteen_bits* ← *max_texts*; { final *link* in module replacement texts }
last_unnamed : *text_pointer* ← 0; { most recent replacement text of unnamed module }

71. ⟨ Set initial values 15 ⟩ +≡
text_link(0) ← 0;

72. If the first byte of a token is less than 200_8, the token occupies a single byte. Otherwise we make a sixteen-bit token by combining two consecutive bytes a and b. If $200_8 \le a < 250_8$, then $(a - 200_8) \times 2^8 + b$ points to an identifier; if $250_8 \le a < 320_8$, then $(a - 250_8) \times 2^8 + b$ points to a module name; otherwise, i.e., if $320_8 \le a < 400_8$, then $(a - 320_8) \times 2^8 + b$ is the number of the module in which the current replacement text appears.

Codes less than 200_8 are 7-bit ASCII codes that represent themselves. In particular, a single-character identifier like 'x' will be a one-byte token, while all longer identifiers will occupy two bytes.

Some of the 7-bit ASCII codes will not be present, however, so we can use them for special purposes. The following symbolic names are used:

param denotes insertion of a parameter. This occurs only in the replacement texts of parametric macros, outside of single-quoted strings in those texts.
begin_comment denotes `@{`, which will become `--`.
end_comment denotes `@}`, which will become a new line.
character_tok denotes the `'` that encloses an character constant.
ascii_tok denotes the `@'` that precedes a ASCII code constant.
output_tok denotes the `@~` that precedes an output file-name.
join denotes the concatenation of adjacent items with no space or line breaks allowed between them (the `@&` operation of `AWEB`).
double_dot denotes '`..`' in ADA.
verbatim denotes the `@=` that begins a verbatim ADA string. It is also used for the end of the string.
force_line denotes the `@\` that forces a new line in the ADA output.

⟨ Types and objects visible from *data_structures* 10 ⟩ +≡
param : **constant** *eight_bits* ← 0; { ASCII null code will not appear }
verbatim : **constant** *eight_bits* ← 2; { extended ASCII alpha should not appear }
force_line : **constant** *eight_bits* ← 3; { extended ASCII beta should not appear }
begin_comment : **constant** *eight_bits* ← 11_8; { ASCII tab mark will not appear }
end_comment : **constant** *eight_bits* ← 12_8; { ASCII line feed will not appear }
character_tok : **constant** *eight_bits* ← 14_8; { ASCII form feed will not appear }
ascii_tok : **constant** *eight_bits* ← 15_8; { ASCII carriage return will not appear }
output_tok : **constant** *eight_bits* ← 16_8; { ASCII `DLE` will not appear }
double_dot : **constant** *eight_bits* ← 40_8; { ASCII space will not appear except in strings }
join : **constant** *eight_bits* ← 177_8; { ASCII delete will not appear }

73. ⟨ Specification of procedures visible from *data_structures* 47 ⟩ +≡
procedure *store_two_bytes*(x : *sixteen_bits*);

74. The following procedure is used to enter a two-byte value into *tok_mem* when a replacement text is being generated.

⟨ Procedures in *data_structures* 48 ⟩ +≡
procedure *store_two_bytes*(x : *sixteen_bits*) **is** { stores high byte, then low byte }
begin
  if *tok_ptr*(z) + 2 > *max_toks* **then**
    overflow("token");
  end if;
  tok_mem(z, *tok_ptr*(z)) ← $x/400_8$; { this could be done by a shift command }
  tok_mem(z, *tok_ptr*(z) + 1) ← x **mod** 400_8; { this could be done by a logical and }
  tok_ptr(z) ← *tok_ptr*(z) + 2;
end *store_two_bytes*;

75. Stacks for output. Let's make sure that our data structures contain enough information to produce the entire ADA program as desired, by working next on the algorithms that actually do produce that program.

76. The output process uses a stack to keep track of what is going on at different "levels" as the macros are being expanded. Entries on this stack have five parts:

end_field is the *tok_mem* location where the replacement text of a particular level will end;
byte_field is the *tok_mem* location from which the next token on a particular level will be read;
name_field points to the name corresponding to a particular level;
repl_field points to the replacement text currently being read at a particular level.
mod_field is the module number, or zero if this is a macro.

The current values of these five quantities are referred to quite frequently, so they are stored in a separate place instead of in the *stack* array. We call the current values *cur_end*, *cur_byte*, *cur_name*, *cur_repl*, and *cur_mod*.

The global variable *stack_ptr* tells how many levels of output are currently in progress. The end of all output occurs when the stack is empty, i.e., when *stack_ptr* = 0.

```
⟨ Types and objects visible from data_structures 10 ⟩ +≡
  subtype mod_range is INTEGER range 0 .. 27777_8;
  type output_state is
    record
      end_field : sixteen_bits;  { ending location of replacement text }
      byte_field : sixteen_bits;  { present location within replacement text }
      name_field : name_pointer;  { byte_start index for text being output }
      repl_field : text_pointer;  { tok_start index for text being output }
      mod_field : mod_range;  { module number or zero if not a module }
    end record;
  cur_state : output_state;  { cur_end, cur_byte, cur_name, cur_repl }
  stack : array (1 .. stack_size) of output_state;  { info for non-current levels }
  stack_ptr : INTEGER range 0 .. stack_size;  { first unused location in the output state stack }
  cur_end : sixteen_bits renames cur_state.end_field;  { current ending location in tok_mem }
  cur_byte : sixteen_bits renames cur_state.byte_field;  { location of next output byte in tok_mem }
  cur_name : name_pointer renames cur_state.name_field;  { pointer to current name being expanded }
  cur_repl : text_pointer renames cur_state.repl_field;  { pointer to current replacement text }
  cur_mod : mod_range renames cur_state.mod_field;  { current module number being expanded }
```

77. It is convenient to keep a global variable *zo* equal to *cur_repl* **mod** *zz*.

```
⟨ Types and objects visible from data_structures 10 ⟩ +≡
  zo : z_range;  { the segment of tok_mem from which output is coming }
```

78. Parameters must also be stacked. They are placed in *tok_mem* just above the other replacement texts, and dummy parameter 'names' are placed in *byte_start* just after the other names. The variables *text_ptr* and *tok_ptr*(*z*) essentially serve as parameter stack pointers during the output phase, so there is no need for a separate data structure to handle this problem.

79. There is an implicit stack corresponding to meta-comments that are output via `@{` and `@}`. But this stack need not be represented in detail, because we only need to know whether it is empty or not. A global variable *brace_level* tells how many items would be on this stack if it were present.

```
⟨ Types and objects visible from data_structures 10 ⟩ +≡
  brace_level : eight_bits;  { current depth of @{...@} nesting }
```

80. To get the output process started, we will perform the following initialization steps. We may assume that *text_link*(0) is nonzero, since it points to the ADA text in the first unnamed module that generates code; if there are no such modules, there is nothing to output, and an error message will have been generated before we do any of the initialization.

```
⟨Initialize the output stacks 80⟩ ≡
  stack_ptr ← 1; brace_level ← 0; cur_name ← 0; cur_repl ← text_link(0); zo ← cur_repl mod zz;
  cur_byte ← tok_start(cur_repl); cur_end ← tok_start(cur_repl + zz); cur_mod ← 0;
```

This code is used in section 100.

81. When the replacement text for name p is to be inserted into the output, the following subroutine is called to save the old level of output and get the new one going.

```
⟨Procedures in output_phase 81⟩ ≡
  procedure push_level(p : name_pointer) is   { suspends the current level }
  begin
    if stack_ptr = stack_size then
      overflow("stack");
    else
      stack(stack_ptr) ← cur_state;   { save cur_end, cur_byte, etc. }
      incr(stack_ptr); cur_name ← p; cur_repl ← equiv(p); zo ← cur_repl mod zz;
      cur_byte ← tok_start(cur_repl); cur_end ← tok_start(cur_repl + zz); cur_mod ← 0;
    end if;
  end push_level;
```

See also sections 82, 84, 94, 96, 98, 101, and 114.

This code is used in section 6*.

82. When we come to the end of a replacement text, the *pop_level* subroutine does the right thing: It either moves to the continuation of this replacement text or returns the state to the most recently stacked level. Part of this subroutine, which updates the parameter stack, will be given later when we study the parameter stack in more detail.

```
⟨Procedures in output_phase 81⟩ +≡
  procedure pop_level is   { do this when cur_byte reaches cur_end }
  begin
    if text_link(cur_repl) = 0 then   { end of macro expansion }
      if ilk(cur_name) = parametric then
        ⟨Remove a parameter from the parameter stack 88⟩;
      end if;
    elsif text_link(cur_repl) < module_flag then   { link to a continuation }
      cur_repl ← text_link(cur_repl);   { we will stay on the same level }
      zo ← cur_repl mod zz; cur_byte ← tok_start(cur_repl); cur_end ← tok_start(cur_repl + zz);
      return;
    end if;
    decr(stack_ptr);   { we will go down to the previous level }
    if stack_ptr > 0 then
      cur_state ← stack(stack_ptr); zo ← cur_repl mod zz;
    end if;
  end pop_level;
```

83. The heart of the output procedure is the *get_output* routine, which produces the next token of output that is not a reference to a macro. This procedure handles all the stacking and unstacking that is necessary. It returns the value *module_number* if the next output begins or ends the replacement text of some module, in which case *cur_val* is that module's number (if beginning) or the negative of that value (if ending). The procedure also returns the value *identifier* if the next output is an identifier of length two or more, in which case *cur_val* points to that identifier name.

```
⟨ Types and objects visible from data_structures 10 ⟩ +≡
  module_number : constant sixteen_bits ← 201_8;  { code returned by get_output for module numbers }
  identifier : constant sixteen_bits ← 202_8;  { code returned by get_output for identifiers }
  cur_val : INTEGER;  { additional information corresponding to output token }
```

84. If *get_output* finds that no more output remains, it returns the value zero.

```
⟨ Procedures in output_phase 81 ⟩ +≡
  function get_output return sixteen_bits is  { returns next token after macro expansion }
    a : sixteen_bits;  { value of current byte }
    b : eight_bits;  { byte being copied }
    bal : sixteen_bits;  { excess of ( versus ) while copying a parameter }
    k : byte_index;  { index into byte_mem }
    w : w_range;  { segment of byte_mem }
  begin
  ⟪restart⟫ if stack_ptr = 0 then
      return 0;
    end if;
    if cur_byte = cur_end then
      cur_val ← −cur_mod; pop_level;
      if cur_val = 0 then
        goto restart;
      end if;
      return module_number;
    end if;
    a ← tok_mem(zo, cur_byte); incr(cur_byte);
    if a < 200_8 then  { one-byte token }
      if a = param then
        ⟨ Start scanning current macro parameter, goto restart 89 ⟩;
      else
        return a;
      end if;
    end if;
    a ← (a − 200_8) ∗ 400_8 + tok_mem(zo, cur_byte); incr(cur_byte);
    if a < 24000_8 then  { 24000_8 = (250_8 − 200_8) ∗ 400_8 }
      ⟨ Expand macro a and return, or goto restart if no output found 86 ⟩;
    end if;
    if a < 50000_8 then  { 50000_8 = (320_8 − 200_8) ∗ 400_8 }
      ⟨ Expand module a − 24000_8, goto restart 85 ⟩;
    end if;
    cur_val ← a − 50000_8; cur_mod ← cur_val; return module_number;
  end get_output;
```

85. The user may have forgotten to give any ADA text for a module name, or the ADA text may have been associated with a different name by mistake.

⟨ Expand module $a - 24000_8$, **goto** *restart* 85 ⟩ ≡
  $a \leftarrow a - 24000_8$;
  if $equiv(a) \neq 0$ **then**
    $push_level(a)$;
  elsif $a \neq 0$ **then**
    $print_nl$("!␣Not␣present:␣<"); $print_id(a)$; $print$(">"); $error$;
  end if;
  goto *restart*

This code is used in section 84.

86. ⟨ Expand macro a and **return**, or **goto** *restart* if no output found 86 ⟩ ≡
  case $ilk(a)$ **is**
    when *normal* ⇒
      $cur_val \leftarrow a$; **return** *identifier*;
    when *simple* ⇒
      $push_level(a)$; **goto** *restart*;
    when *parametric* ⇒
      ⟨ Put a parameter on the parameter stack, or **goto** *restart* if error occurs 87 ⟩;
      $push_level(a)$; **goto** *restart*;
    when others ⇒
      $confusion$("output");
  end case;
  return a

This code is used in section 84.

87. We come now to the interesting part, the job of putting a parameter on the parameter stack. First we pop the stack if necessary until getting to a level that hasn't ended. Then the next character must be a '('; and since parentheses are balanced on each level, the entire parameter must be present, so we can copy it without difficulty.

⟨ Put a parameter on the parameter stack, or **goto** *restart* if error occurs 87 ⟩ ≡
  while $(cur_byte = cur_end) \wedge (stack_ptr > 0)$ **loop**
    pop_level;
  end loop;
  if $(stack_ptr = 0) \vee (tok_mem(zo, cur_byte) \neq$ [(]) **then**
    $print_nl$("!␣No␣parameter␣given␣for␣"); $print_id(a)$; $error$; **goto** *restart*;
  end if;
  ⟨ Copy the parameter into tok_mem 90 ⟩;
  $equiv(name_ptr) \leftarrow text_ptr$; $ilk(name_ptr) \leftarrow simple$; $w \leftarrow name_ptr$ **mod** ww; $k \leftarrow byte_ptr(w)$;
  if $name_ptr > max_names - ww$ **then**
    $overflow$("name");
  end if;
  $byte_start(name_ptr + ww) \leftarrow k$; $incr(name_ptr)$;
  if $text_ptr > max_texts - zz$ **then**
    $overflow$("text");
  end if;
  $text_link(text_ptr) \leftarrow 0$; $tok_start(text_ptr + zz) \leftarrow tok_ptr(z)$; $incr(text_ptr)$; $z \leftarrow text_ptr$ **mod** zz

This code is used in section 86.

88. The *pop_level* routine undoes the effect of parameter-pushing when a parameter macro is finished:

⟨ Remove a parameter from the parameter stack 88 ⟩ ≡
  decr(*name_ptr*); *decr*(*text_ptr*); $z \leftarrow$ *text_ptr* **mod** *zz*;
  stat
    if *tok_ptr*(*z*) > *max_tok_ptr*(*z*) **then**
      max_tok_ptr(*z*) ← *tok_ptr*(*z*);
    end if;
  tats { the maximum value of *tok_ptr* occurs just before parameter popping }
  tok_ptr(*z*) ← *tok_start*(*text_ptr*)

This code is used in section 82.

89. When a parameter occurs in a replacement text, we treat it as a simple macro in position (*name_ptr* − 1):

⟨ Start scanning current macro parameter, **goto** *restart* 89 ⟩ ≡
  push_level(*name_ptr* − 1); **goto** *restart*

This code is used in section 84.

90. Similarly, a *param* token encountered as we copy a parameter is converted into a simple macro call for $name_ptr - 1$. Some care is needed to handle cases like *macro*(`#`; *print*(`"#)"`)); the `#` token will have been changed to *param* outside of strings, but we still must distinguish 'real' parentheses from those in strings.

```
define app_repl(#) ≡
        if tok_ptr(z) = max_toks then
          overflow("token");
        end if;
        tok_mem(z, tok_ptr(z)) ← #; incr(tok_ptr(z))

⟨ Copy the parameter into tok_mem 90 ⟩ ≡
  bal ← 1; incr(cur_byte);   { skip the opening '(' }
  loop
    b ← tok_mem(zo, cur_byte); incr(cur_byte);
    if b = param then
      store_two_bytes(name_ptr + 77777_8);
    else
      if b ≥ 200_8 then
        app_repl(b); b ← tok_mem(zo, cur_byte); incr(cur_byte);
      else
        case b is
          when [(] ⇒
            incr(bal);
          when [)] ⇒
            decr(bal); exit when bal = 0;
          when character_tok | ascii_tok ⇒
            app_repl(b); b ← tok_mem(zo, cur_byte); incr(cur_byte);
          when ["] ⇒
            repeat
              app_repl(b); b ← tok_mem(zo, cur_byte); incr(cur_byte);
              if b = ["] ∧ tok_mem(zo, cur_byte) = ["] then
                app_repl(["]); app_repl(["]); b ← tok_mem(zo, cur_byte + 1); cur_byte ← cur_byte + 2;
              end if;
            until (b = ["]);   { copy string, don't change bal }
          when others ⇒
            null;
        end case;
      end if;
      app_repl(b);
    end if;
  end loop
```

This code is used in section 87.

91. Producing the output. The *get_output* routine above handles most of the complexity of output generation, but there are two further considerations that have a nontrivial effect on ATANGLE's algorithms.

First, we want to make sure that the output is broken into lines not exceeding *line_length* characters per line, where these breaks occur at valid places (e.g., not in the middle of a string or a constant or an identifier, not between '/' and '=', not at a '@&' position where quantities are being joined together). Therefore we assemble the output into a buffer before deciding where the line breaks will appear. However, we make very little attempt to make "logical" line breaks that would enhance the readability of the output; people are supposed to read the input of ATANGLE or the TEXed output of AWEAVE, but not the tangled-up output. The only concession to readability is that a break after a semicolon will be made if possible, since commonly used "pretty printing" routines give better results in such cases.

The following solution to these problems has been adopted: An array *out_buf* contains characters that have been generated but not yet output, and there are three pointers into this array. One of these, *out_ptr*, is the number of characters currently in the buffer, and we will have $1 \le out_ptr \le line_length$ most of the time. The second is *break_ptr*, which is the largest value $\le out_ptr$ such that we are definitely entitled to end a line by outputting the characters $out_buf(1 \,..\, (break_ptr - 1))$; we will always have $break_ptr \le line_length$. Finally, *semi_ptr* is either zero or the largest known value of a legal break after a semicolon on the current line; we will always have $semi_ptr \le break_ptr$.

⟨ Types and objects visible from *data_structures* 10 ⟩ +≡
  subtype *out_buffer_index* **is** INTEGER **range** 0 .. *out_buf_size*;
  out_buf : *buffer_type*(*out_buffer_index*); { assembled characters }
  out_ptr : *out_buffer_index*; { first available place in *out_buf* }
  break_ptr : *out_buffer_index*; { last breaking place in *out_buf* }
  semi_ptr : *out_buffer_index*; { last semicolon breaking place in *out_buf* }

92. Besides having those three pointers, the output process is in one of several states:

num_or_id means that the last item in the buffer is a number or identifier, hence a blank space or line break must be inserted if the next item is also a number or identifier.

unbreakable means that the last item in the buffer was followed by the @& operation that inhibits spaces between it and the next item.

misc means none of the above.

In all states the last item in the buffer lies between *break_ptr* and $out_ptr - 1$, inclusive.

Beside the states there is a boolean variable, called *comment_output*, which is set true, if the material to be output is part of the meta-comments between '@{' and '@}' and therefore has to be preceded by '--'.

⟨ Types and objects visible from *data_structures* 10 ⟩ +≡
  misc : **constant** *eight_bits* ← 0; { state associated with special characters }
  num_or_id : **constant** *eight_bits* ← 1; { state associated with numbers and identifiers }
  unbreakable : **constant** *eight_bits* ← 2; { state associated with @& }
  out_state : *eight_bits*; { current status of partial output }
  comment_output : BOOLEAN; { outputting meta-comment? }

93. During the output process, *line* will equal the number of the next line to be output.

⟨ Initialize the output buffer 93 ⟩ ≡
  out_state ← *misc*; *out_ptr* ← 0; *break_ptr* ← 0; *semi_ptr* ← 0; *out_buf*(0) ← 0; *line* ← 1;
  comment_output ← FALSE;

This code is used in section 100.

94. Here is a routine that is invoked when *out_ptr* > *line_length* or when it is time to flush out the final line. The *flush_buffer* procedure often writes out the line up to the current *break_ptr* position, then moves the remaining information to the front of *out_buf*. However, it prefers to write only up to *semi_ptr*, if the residual line won't be too long.

```
define check_break ≡
        if out_ptr > line_length then
          flush_buffer; end if
⟨Procedures in output_phase 81⟩ +≡
  procedure flush_buffer is   { writes one line to output file }
    b : out_buffer_index ← break_ptr;   { value of break_ptr upon entry }
  begin
    if comment_output then
      TEXT_IO.PUT(ada_file, "--");
    end if;
    if (semi_ptr ≠ 0) ∧ (out_ptr − semi_ptr ≤ line_length) then
      break_ptr ← semi_ptr;
    end if;
    for k ∈ 1 .. break_ptr loop
      TEXT_IO.PUT(ada_file, xchr(out_buf(k − 1)));
    end loop;
    new_ln(ada_file); incr(line); incr(f_line);
    if line mod 100 = 0 then
      print(".");
      if line mod 500 = 0 then
        print_int(line, 1); update_terminal;   { progress report }
      end if;
    end if;
    if break_ptr < out_ptr then
      if out_buf(break_ptr) = [␣] then
        incr(break_ptr);   { drop space at break }
        if break_ptr > b then
          b ← break_ptr;
        end if;
      end if;
      out_buf(0 .. out_ptr − break_ptr − 1) ← out_buf(break_ptr .. out_ptr − 1);
    end if;
    out_ptr ← out_ptr − break_ptr; break_ptr ← b − break_ptr; semi_ptr ← 0;
    if out_ptr > line_length then
      err_print("!␣Long␣line␣must␣be␣truncated");
      out_ptr ← line_length;
    end if;
  end flush_buffer;
```

95. ⟨Empty the last line from the buffer 95⟩ ≡

```
  break_ptr ← out_ptr; semi_ptr ← 0; flush_buffer;
  if brace_level ≠ 0 then
    new_ln; print("!␣Program␣ended␣at␣brace␣level␣"); print_int(brace_level, 1); error;
  end if;
```

This code is used in section 100.

96. Another simple and useful routine appends the decimal equivalent of a nonnegative integer to the output buffer.

define *app*(#) ≡ *out_buf*(*out_ptr*) ← #; *incr*(*out_ptr*) { append a single character }

⟨ Procedures in *output_phase* 81 ⟩ +≡

```
procedure send_val(v : INTEGER) is   { puts v into buffer, assumes v ≥ 0 }
  k : out_buffer_index ← out_buf_size;   { index into out_buf }
  u : INTEGER ← v;
begin   { first we put the digits at the very end of out_buf }
  repeat
    out_buf(k) ← u mod 10; u ← u/10; decr(k);
  until (u = 0);
  repeat
    incr(k); app(out_buf(k) + [0]);
  until (k = out_buf_size);   { then we append them, most significant first }
end send_val;
```

97. The output states are kept up to date by the *send_out* procedure, which has two parameters: t tells the type of information being sent and v contains the information proper. Some information may also be passed in the array *out_contrib*.

If $t = misc$ then v is a character to be output.
If $t = str$ then v is the length of a string or something like '/=' in *out_contrib*.
If $t = ident$ then v is the length of an identifier in *out_contrib*.
If $t = frac$ then v is the length of a fraction and/or exponent in *out_contrib*.

⟨ Types and objects visible from *data_structures* 10 ⟩ +≡

```
str : constant eight_bits ← 1;   { send_out code for a string }
ident : constant eight_bits ← 2;   { send_out code for an identifier }
frac : constant eight_bits ← 3;   { send_out code for a fraction }
out_contrib : buffer_type(1 .. line_length);   { a contribution to out_buf }
```

98. A slightly subtle point in the following code is that the user may ask for a *join* operation (i.e., `@&`) following whatever is being sent out. We will see later that *join* is implemented in part by calling *send_out*(*frac*, 0).

```
⟨ Procedures in output_phase 81 ⟩ +≡
  procedure send_out(t : eight_bits; v : sixteen_bits) is   { outputs v of type t }
  begin
    ⟨ Get the buffer ready for appending the new information 99 ⟩;
    if t ≠ misc then
      for k ∈ 1 .. v loop
        app(out_contrib(k));
      end loop;
    else
      app(v);
    end if;
    check_break;
    if (t = misc) ∧ (v = [;]) then
      semi_ptr ← out_ptr; break_ptr ← out_ptr;
    end if;
    if t ≥ ident then
      out_state ← num_or_id;   { t = ident or frac }
    else
      out_state ← misc;   { t = str or misc }
    end if;
  end send_out;
```

99.

```
⟨ Get the buffer ready for appending the new information 99 ⟩ ≡
⟪restart⟫ case out_state is
    when num_or_id ⇒
      if t ≠ frac then
        break_ptr ← out_ptr;
        if t = ident then
          app([␣]);
        end if;
      end if;
    when misc ⇒
      if t ≠ frac then
        break_ptr ← out_ptr;
      end if;
    when others ⇒
      null;   { this is for unbreakable state }
  end case
```

This code is used in section 98.

100. The big output switch. To complete the output process, we need a routine that takes the results of *get_output* and feeds them to *send_out* or *send_val*. This procedure '*send_the_output*' will be invoked just once, as follows:

```
⟨ Phase II: Output the contents of the compressed tables 100 ⟩ ≡
  if text_link(0) = 0 then
    print_nl("!␣No␣output␣was␣specified."); mark_harmless;
  else
    print_nl("Writing␣the␣output␣file"); update_terminal;
    ⟨ Initialize the output stacks 80 ⟩
    ⟨ Initialize the output buffer 93 ⟩
    send_the_output;
    ⟨ Empty the last line from the buffer 95 ⟩
    print_nl("Done.");
  end if
```

This code is used in section 114.

101. A many-way switch is used to send the output:

```
⟨ Procedures in output_phase 81 ⟩ +≡
  procedure send_the_output is
    cur_char : eight_bits;  { the latest character received }
    k : INTEGER range 0 .. line_length;  { index into out_contrib }
    j : byte_index;  { index into byte_mem }
    w : w_range;  { segment of byte_mem }
  begin
    while stack_ptr > 0 loop
      cur_char ← get_output;
    ⟪reswitch⟫ case cur_char is
        when 0 ⇒ null;  { this case might arise if output ends unexpectedly }
        ⟨ Cases related to identifiers 104 ⟩
        ⟨ Cases related to constants, possibly leading to get_fraction or reswitch 110 ⟩
        ⟨ Cases like /= and := 102 ⟩
        when ["] ⇒ ⟨ Send a string, goto reswitch 105 ⟩;
        when ⟨ Other printable characters 103 ⟩ ⇒ send_out(misc, cur_char);
        ⟨ Cases involving @{ and @} 112 ⟩
        when join ⇒ send_out(frac, 0); out_state ← unbreakable;
        when verbatim ⇒ ⟨ Send verbatim string 106 ⟩;
        when output_tok ⇒ ⟨ Get the file_name and redirect further output 108 ⟩;
        when force_line ⇒ ⟨ Force a line break 113 ⟩;
        when others ⇒
          new_ln; print("!␣Can´t␣output␣ASCII␣code␣"); print_int(cur_char, 1); error;
      end case;
      goto continue;
    ⟪get_fraction⟫ ⟨ Special code to finish real constants 111 ⟩;
    ⟪continue⟫ null;
    end loop;
  end send_the_output;
```

102. ⟨ Cases like `/=` and `:=` 102 ⟩ ≡

```
when and_sign ⇒ out_contrib(1 .. 3) ← ([A], [N], [D]); send_out(ident, 3);
when not_sign ⇒ out_contrib(1 .. 3) ← ([N], [O], [T]); send_out(ident, 3);
when set_element_sign ⇒ out_contrib(1 .. 2) ← ([I], [N]); send_out(ident, 2);
when or_sign ⇒ out_contrib(1 .. 2) ← ([O], [R]); send_out(ident, 2);
when left_arrow ⇒ out_contrib(1 .. 2) ← ([:], [=]); send_out(str, 2);
when not_equal ⇒ out_contrib(1 .. 2) ← ([/], [=]); send_out(str, 2);
when less_or_equal ⇒ out_contrib(1 .. 2) ← ([<], [=]); send_out(str, 2);
when greater_or_equal ⇒ out_contrib(1 .. 2) ← ([>], [=]); send_out(str, 2);
when equivalence_sign ⇒ out_contrib(1 .. 2) ← ([=], [=]); send_out(str, 2);
when double_dot ⇒ out_contrib(1 .. 2) ← ([.], [.]); send_out(str, 2);
when right_arrow ⇒ out_contrib(1 .. 2) ← ([=], [>]); send_out(str, 2);
when box_sign ⇒ out_contrib(1 .. 2) ← ([<], [>]); send_out(str, 2);
when left_label_bracket ⇒ out_contrib(1 .. 2) ← ([<], [<]); send_out(str, 2);
when right_label_bracket ⇒ out_contrib(1 .. 2) ← ([>], [>]); send_out(str, 2);
when double_star ⇒ out_contrib(1 .. 2) ← ([*], [*]); send_out(str, 2);
when character_tok ⇒ out_contrib(1 .. 3) ← ([´], get_output, [´]); send_out(str, 3);
when ascii_tok ⇒
  if out_state = num_or_id then
    send_out(misc, [␣]);
  end if;
  send_val(get_output); out_state ← num_or_id;
```

This code is used in section 101.

103. Please don't ask how all of the following characters can actually get through `ATANGLE` outside of strings. It seems that [{] cannot actually occur at this point of the program, but it has been included just in case `ATANGLE` changes.

⟨ Other printable characters 103 ⟩ ≡

```
[!] | [#] | [$] | [%] | [&] | [´] | [(] | [)] | [*] | [,] | [/] | [:] | [;] | [<] | [=] | [>] | [?] | [@] | [[] | [\] | []] |
    [^] | [_] | [`] | [{] | [|] | [+] | [-]
```

This code is used in section 101.

104. Single-character identifiers represent themselves, while longer ones appear in *byte_mem*. All must be converted to uppercase, with underlines removed. Extremely long identifiers must be chopped.

⟨ Cases related to identifiers 104 ⟩ ≡

```
when [A] .. [Z] ⇒ out_contrib(1) ← cur_char; send_out(ident, 1);
when [a] .. [z] ⇒ out_contrib(1) ← cur_char − 40₈; send_out(ident, 1);
when identifier ⇒ k ← 0; j ← byte_start(cur_val); w ← cur_val mod ww;
  while (k < max_id_length) ∧ (j < byte_start(cur_val + ww)) loop
    incr(k); out_contrib(k) ← byte_mem(w, j); incr(j);
    if out_contrib(k) ≥ [a] then
      out_contrib(k) ← out_contrib(k) − 40₈;
    end if;
  end loop;
  send_out(ident, k);
```

This code is used in section 101.

105. After sending a string, we need to look ahead at the next character, in order to see if there were two consecutive single-quote marks. Afterwards we go to *reswitch* to process the next character.

```
⟨ Send a string, goto reswitch 105 ⟩ ≡
  k ← 1; out_contrib(1) ← ["];
  repeat
    if k < line_length then
      incr(k);
    end if;
    out_contrib(k) ← get_output;
  until ((out_contrib(k) = ["]) ∨ (stack_ptr = 0));
  if k = line_length then
    err_print("!␣String␣too␣long");
  end if;
  send_out(str, k); cur_char ← get_output;
  if cur_char = ["] then
    out_state ← unbreakable;
  end if;
  goto reswitch
```

This code is used in section 101.

106. Sending a verbatim string is similar, but we don't have to look ahead.

```
⟨ Send verbatim string 106 ⟩ ≡
  k ← 0;
  repeat
    if k < line_length then
      incr(k);
    end if;
    out_contrib(k) ← get_output;
  until ((out_contrib(k) = verbatim) ∨ (stack_ptr = 0));
  if k = line_length then
    err_print("!␣Verbatim␣string␣too␣long");
  end if;
  send_out(str, k − 1)
```

This code is used in section 101.

107. Getting the new output file name is similar to sending a verbatim string; if there is a legal name, the file will be created and the variables *file_name* and *fn_length* will be set accordingly and *f_line* is set to 1.

```
⟨ Types and objects visible from data_structures 10 ⟩ +≡
  file_name : STRING(1 . . 80);
  fn_length : INTEGER range 1 . . 80;
  f_line : INTEGER;
```

108. Before we redirect output we will empty the output buffer.

```
⟨ Get the file_name and redirect further output 108 ⟩ ≡
  ⟨ Force a line break 113 ⟩;
  k ← 0;
  loop
    cur_char ← get_output; exit when (cur_char = output_tok) ∨ (stack_ptr = 0);
    incr(k); file_name(k) ← xchr(cur_char);
  end loop;
  if k > 0 then
    TEXT_IO.CLOSE(ada_file); fn_length ← k;
    if ¬create_ada_file(file_name(1 .. k)) then
      fatal_error("!␣Could␣not␣open␣file␣`" & file_name(1 .. k) & "´.");
    end if;
    f_line ← 1;
  else
    err_print("!␣File␣name␣should␣have␣length␣>␣0");
  end if
```

This code is used in section 101.

109. In the cases related to numeric literals there is some care needed, because of the "based literal", which ADA supports; a global variable *scanning_based_number* tells whether there is more to do.

```
⟨ Types and objects visible from data_structures 10 ⟩ +≡
  scanning_based_number : BOOLEAN ← FALSE;
```

110. ⟨ Cases related to constants, possibly leading to *get_fraction* or *reswitch* 110 ⟩ ≡

```
when [0] .. [9] ⇒
  scanning_based_number ← FALSE; k ← 0;
  loop
    if cur_char ≠ [_] then
      incr(k); out_contrib(k) ← cur_char; cur_char ← get_output;
      if cur_char = [#] then
        scanning_based_number ← ¬scanning_based_number;
      else
        exit when ((¬scanning_based_number) ∧ (cur_char ∉ [0] .. [9]) ∧ (cur_char ≠ [_]));
      end if;
    else
      cur_char ← get_output;
    end if;
  end loop;
  send_out(ident, k); k ← 0;
  if cur_char = [e] then
    cur_char ← [E];
  end if;
  if cur_char = [E] then
    goto get_fraction;
  else
    goto reswitch;
  end if;
when [.] ⇒
  k ← 1; out_contrib(1) ← [.]; cur_char ← get_output;
  if cur_char = [.] then
    out_contrib(2) ← [.]; send_out(str, 2);
  elsif cur_char ∈ [0] .. [9] then
    goto get_fraction;
  else
    send_out(misc, [.]); goto reswitch;
  end if;
```

This code is used in section 101.

111. The following code appears at label '*get_fraction*', when we want to scan to the end of a real constant. The first k characters of a fraction have already been placed in *out_contrib*, and *cur_char* is the next character.

```
⟨ Special code to finish real constants 111 ⟩ ≡
  repeat
    if k < line_length then
      incr(k);
    end if;
    out_contrib(k) ← cur_char; cur_char ← get_output;
    if (out_contrib(k) = [E]) ∧ ((cur_char = [+]) ∨ (cur_char = [-])) then
      if k < line_length then
        incr(k);
      end if;
      out_contrib(k) ← cur_char; cur_char ← get_output;
    elsif cur_char = [e] then
      cur_char ← [E];
    end if;
  until ((cur_char ≠ [E]) ∧ (cur_char ∉ [0] .. [9]));
  if k = line_length then
    err_print("!␣Fraction␣too␣long");
  end if;
  send_out(frac, k); goto reswitch
```

This code is used in section 101.

112. Here the variable *comment_output* is set corresponding to the value of *brace_level*; further after each comment a line break is inserted.

```
⟨ Cases involving @{ and @} 112 ⟩ ≡
when begin_comment ⇒
  ⟨ Force a line break 113 ⟩;
  incr(brace_level); comment_output ← TRUE;
when end_comment ⇒
  if brace_level > 0 then
    decr(brace_level);
    if brace_level = 0 then
      ⟨ Force a line break 113 ⟩;
      comment_output ← FALSE;
    end if;
  else
    err_print("!␣Extra␣@}");
  end if;
when module_number ⇒
  semi_ptr ← 0; out_contrib(1 . . 2) ← ([-], [-]); send_out(str, 2);
  while cur_char = module_number loop
    out_state ← unbreakable; send_out(misc, [{]); out_state ← unbreakable;
    if cur_val < 0 then
      send_out(misc, [:]); out_state ← unbreakable; send_val(−cur_val);
    else
      send_val(cur_val); out_state ← unbreakable; send_out(misc, [:]);
    end if;
    out_state ← unbreakable; send_out(misc, [}]); cur_char ← get_output;
  end loop;
  ⟨ Force a line break 113 ⟩;
  goto reswitch;
```

This code is used in section 101.

113. ⟨ Force a line break 113 ⟩ ≡

```
  send_out(str, 0);  { normalize the buffer }
  while out_ptr > 0 loop
    if out_ptr ≤ line_length then
      break_ptr ← out_ptr;
    end if;
    flush_buffer;
  end loop;
  out_state ← misc
```

This code is used in sections 101, 108, and 112(3).

114. ⟨ Procedures in *output_phase* 81 ⟩ +≡

```
  procedure phase_ii is
  begin
    ⟨ Phase II: Output the contents of the compressed tables 100 ⟩;
  end phase_ii;
```

115. Introduction to the input phase. We have now seen that ATANGLE will be able to output the full ADA program, if we can only get that program into the byte memory in the proper format. The input process is something like the output process in reverse, since we compress the text as we read it in and we expand it as we write it out.

There are three main input routines. The most interesting is the one that gets the next token of an ADA text; the other two are used to scan rapidly past TEX text in the AWEB source code. One of the latter routines will jump to the next token that starts with '@', and the other skips to the end of an ADA comment.

116. But first we need to consider the low-level routine *get_line* that takes care of merging *change_file* into *web_file*. The *get_line* procedure also updates the line numbers for error messages.

⟨ Types and objects visible from *data_structures* 10 ⟩ +≡
  line : INTEGER; { the number of the current line in the current file }
  other_line : INTEGER; { the number of the current line in the input file that is not currently being read }
  temp_line : INTEGER; { used when interchanging *line* with *other_line* }
  limit : *buffer_index*; { the last character position occupied in the buffer }
  loc : *buffer_index*; { the next character position to be read from the buffer }
  input_has_ended : BOOLEAN; { if TRUE, there is no more input }
  changing : BOOLEAN; { if TRUE, the current line is from *change_file* }

117. As we change *changing* from TRUE to FALSE and back again, we must remember to swap the values of *line* and *other_line* so that the *err_print* routine will be sure to report the correct line number.

define *change_changing* ≡ *changing* ← ¬*changing*; *temp_line* ← *other_line*; *other_line* ← *line*;
  line ← *temp_line* { *line* ↔ *other_line* }

118. When *changing* is FALSE, the next line of *change_file* is kept in *change_buffer*(0 .. *change_limit*), for purposes of comparison with the next line of *web_file*. After the change file has been completely input, we set *change_limit* ← 0, so that no further matches will be made.

⟨ Types and objects visible from *data_structures* 10 ⟩ +≡
  change_buffer : *buffer_type*(*buffer_index*);
  change_limit : *buffer_index*; { the last position occupied in *change_buffer* }

119. Here's a simple function that checks if the two buffers are different.

⟨ Procedures in *input_phase* 119 ⟩ ≡
```
function lines_dont_match return BOOLEAN is
begin
  if change_limit ≠ limit then
    return TRUE;
  end if;
  if limit > 0 and then change_buffer(0 .. limit − 1) ≠ buffer(0 .. limit − 1) then
    return TRUE;
  else
    return FALSE;
  end if;
end lines_dont_match;
```
See also sections 120, 124, 127, 132, 133, 134, 137, 149, 155, 157, and 164.

This code is used in section 7*.

120. Procedure *prime_the_change_buffer* sets *change_buffer* in preparation for the next matching operation. Since blank lines in the change file are not used for matching, we have $(change_limit = 0) \wedge \neg changing$ if and only if the change file is exhausted. This procedure is called only when *changing* is true; hence error messages will be reported correctly.

```
⟨ Procedures in input_phase 119 ⟩ +≡
  procedure prime_the_change_buffer is
  begin
    change_limit ← 0;  { this value will be used if the change file ends }
    ⟨ Skip over comment lines in the change file; return if end of file 121 ⟩;
    ⟨ Skip to the next nonblank line; return if end of file 122 ⟩;
    ⟨ Move buffer and limit to change_buffer and change_limit 123 ⟩;
  end prime_the_change_buffer;
```

121. While looking for a line that begins with `@x` in the change file, we allow lines that begin with `@`, as long as they don't begin with `@y` or `@z` (which would probably indicate that the change file is fouled up).

```
⟨ Skip over comment lines in the change file; return if end of file 121 ⟩ ≡
  loop
    incr(line);
    if ¬input_ln(change_file) then
      return;
    end if;
    if limit < 2 then
      goto continue;
    end if;
    if buffer(0) ≠ [@] then
      goto continue;
    end if;
    if buffer(1) ∈ [X] .. [Z] then
      buffer(1) ← buffer(1) + [z] − [Z];  { lowercasify }
    end if;
    exit when buffer(1) = [x];
    if buffer(1) ∈ [y] .. [z] then
      loc ← 2; err_print("!␣Where␣is␣the␣matching␣@x?");
    end if;
  ⟪continue⟫ null;
  end loop
```

This code is used in section 120.

122. Here we are looking at lines following the `@x`.

```
⟨ Skip to the next nonblank line; return if end of file 122 ⟩ ≡
  repeat
    incr(line);
    if ¬input_ln(change_file) then
      err_print("!␣Change␣file␣ended␣after␣@x");
      return;
    end if;
  until (limit > 0)
```

This code is used in section 120.

123. ⟨ Move *buffer* and *limit* to *change_buffer* and *change_limit* 123 ⟩ ≡

```
change_limit ← limit;
if limit > 0 then
  change_buffer(0 .. limit − 1) ← buffer(0 .. limit − 1);
end if
```

This code is used in sections 120 and 124.

124. The following procedure is used to see if the next change entry should go into effect; it is called only when *changing* is false. The idea is to test whether or not the current contents of *buffer* matches the current contents of *change_buffer*. If not, there's nothing more to do; but if so, a change is called for: All of the text down to the `@y` is supposed to match. An error message is issued if any discrepancy is found. Then the procedure prepares to read the next line from *change_file*.

⟨ Procedures in *input_phase* 119 ⟩ +≡

```
procedure check_change is   { switches to change_file if the buffers match }
  n : INTEGER ← 0;   { the number of discrepancies found }
begin
  if lines_dont_match then
    return;
  end if;
  loop
    change_changing;   { now it's TRUE }
    incr(line);
    if ¬input_ln(change_file) then
      err_print("!␣Change␣file␣ended␣before␣@y");
      change_limit ← 0; change_changing;   { FALSE again }
      return;
    end if;
    ⟨ If the current line starts with @y, report any discrepancies and return 125 ⟩;
    ⟨ Move buffer and limit to change_buffer and change_limit 123 ⟩;
    change_changing;   { now it's FALSE }
    incr(line);
    if ¬input_ln(web_file) then
      err_print("!␣AWEB␣file␣ended␣during␣a␣change");
      input_has_ended ← TRUE; return;
    end if;
    if lines_dont_match then
      incr(n);
    end if;
  end loop;
end check_change;
```

125. ⟨ If the current line starts with `@y`, report any discrepancies and **return** 125 ⟩ ≡

```
if (limit > 1) and then buffer(0) = [@] then
  if buffer(1) ∈ [X] .. [Z] then
    buffer(1) ← buffer(1) + [z] − [Z];  { lowercasify }
  end if;
  if (buffer(1) = [x]) ∨ (buffer(1) = [z]) then
    loc ← 2; err_print("!␣Where␣is␣the␣matching␣@y?");
  elsif buffer(1) = [y] then
    if n > 0 then
      loc ← 2; new_ln; print("!␣Hmm...␣"); print_int(n, 1);
      print("␣of␣the␣preceding␣lines␣failed␣to␣match"); error;
    end if;
    return;
  end if;
end if
```

This code is used in section 124.

126. ⟨ Initialize the input system 126 ⟩ ≡

```
line ← 0; other_line ← 0;
changing ← TRUE; prime_the_change_buffer; change_changing;
limit ← 0; loc ← 1; buffer(0) ← [␣]; input_has_ended ← FALSE
```

This code is used in section 164.

127. The *get_line* procedure is called when *loc* > *limit*; it puts the next line of merged input into the buffer and updates the other variables appropriately. A space is placed at the right end of the line.

⟨ Procedures in *input_phase* 119 ⟩ +≡

```
procedure get_line is   { inputs the next line }
begin
⟨⟨restart⟩⟩ if changing then
      ⟨ Read from change_file and maybe turn off changing 129 ⟩;
  end if;
  if ¬changing then
    ⟨ Read from web_file and maybe turn on changing 128 ⟩;
    if changing then
      goto restart;
    end if;
  end if;
  loc ← 0; buffer(limit) ← [␣];
end get_line;
```

128. ⟨ Read from *web_file* and maybe turn on *changing* 128 ⟩ ≡

```
incr(line);
if ¬input_ln(web_file) then
  input_has_ended ← TRUE;
elsif (limit = change_limit) and then ((buffer(0) = change_buffer(0)) and then (change_limit > 0))
        then
  check_change;
end if
```

This code is used in section 127.

129. ⟨ Read from *change_file* and maybe turn off *changing* 129 ⟩ ≡

```
incr(line);
if ¬input_ln(change_file) then
  err_print("!␣Change␣file␣ended␣without␣@z");
  buffer(0 .. 1) ← ([@], [z]); limit ← 2;
end if;
if (limit > 1) and then (buffer(0) = [@]) then   { check if the change has ended }
  if buffer(1) ∈ [X] .. [Z] then
    buffer(1) ← buffer(1) + [z] − [Z];   { lowercasify }
  end if;
  if buffer(1) ∈ [x] .. [y] then
    loc ← 2; err_print("!␣Where␣is␣the␣matching␣@z?");
  elsif buffer(1) = [z] then
    prime_the_change_buffer; change_changing;
  end if;
end if
```

This code is used in section 127.

130. At the end of the program, we will tell the user if the change file had a line that didn't match any relevant line in *web_file*.

⟨ Check that all changes have been read 130 ⟩ ≡

```
if change_limit ≠ 0 then   { changing is false }
  buffer(0 .. change_limit) ← change_buffer(0 .. change_limit); limit ← change_limit; changing ← TRUE;
  line ← other_line; loc ← change_limit; err_print("!␣Change␣file␣entry␣did␣not␣match");
end if
```

This code is used in section 167.

131. Important milestones are reached during the input phase when certain control codes are sensed.

Control codes in AWEB begin with '@', and the next character identifies the code. Some of these are of interest only to AWEAVE, so ATANGLE ignores them; the others are converted by ATANGLE into internal code numbers by the *control_code* function below. The ordering of these internal code numbers has been chosen to simplify the program logic; larger numbers are given to the control codes that denote more significant milestones.

⟨ Types and objects visible from *data_structures* 10 ⟩ +≡

```
ignore : constant eight_bits ← 0;   { control code of no interest to ATANGLE }
control_text : constant eight_bits ← 203_8;   { control code for '@t', '@^', etc. }
format : constant eight_bits ← 204_8;   { control code for '@f' }
definition : constant eight_bits ← 205_8;   { control code for '@d' }
begin_ada : constant eight_bits ← 206_8;   { control code for '@a' }
module_name : constant eight_bits ← 207_8;   { control code for '@<' }
new_module : constant eight_bits ← 210_8;   { control code for '@␣' and '@*' }
```

132. ⟨ Procedures in *input_phase* 119 ⟩ +≡

```
function control_code(c : ASCII_code)return eight_bits is   { convert c after @ }
begin
  case c is
    when [@] ⇒ return [@];   { 'quoted' at sign }
    when [´] ⇒ return ascii_tok;   { precedes ASCII code constant }
    when [~] ⇒ return output_tok;   { precedes output file-name }
    when [␣] | tab_mark ⇒ return new_module;   { beginning of a new module }
    when [*] ⇒ print("*"); print_int(module_count + 1, 1); update_terminal;   { print a progress report }
      return new_module;   { beginning of a new module }
    when [D] | [d] ⇒ return definition;   { macro definition }
    when [F] | [f] ⇒ return format;   { format definition }
    when [{] ⇒ return begin_comment;   { begin-comment delimiter }
    when [}] ⇒ return end_comment;   { end-comment delimiter }
    when [A] | [a] ⇒ return begin_ada;   { ADA text in unnamed module }
    when [T] | [t] | [^] | [i] | [I] | [b] | [B] | [r] | [R] | [p] | [P] | [s] | [S] | [.] | [:] ⇒ return
          control_text;   { control text to be ignored }
    when [&] ⇒ return join;   { concatenate two tokens }
    when [<] ⇒ return module_name;   { beginning of a module name }
    when [=] ⇒ return verbatim;   { beginning of ADA verbatim mode }
    when [\] ⇒ return force_line;   { force a new line in ADA output }
    when others ⇒ return ignore;   { ignore all other cases }
  end case;
end control_code;
```

133. The *skip_ahead* procedure reads through the input at fairly high speed until finding the next non-ignorable control code, which it returns.

⟨ Procedures in *input_phase* 119 ⟩ +≡

```
function skip_ahead return eight_bits is   { skip to next control code }
  c : eight_bits;   { control code found }
begin
  loop
    if loc > limit then
      get_line;
      if input_has_ended then
        return new_module;
      end if;
    end if;
    buffer(limit + 1) ← [@];
    while buffer(loc) ≠ [@] loop
      incr(loc);
    end loop;
    if loc ≤ limit then
      loc ← loc + 2; c ← control_code(buffer(loc − 1));
      if (c ≠ ignore) ∨ (buffer(loc − 1) = [>]) then
        return c;
      end if;
    end if;
  end loop;
end skip_ahead;
```

134. The *skip_comment* procedure reads through the input at somewhat high speed until finding the first unmatched right brace or until coming to the end of the file. It ignores characters following '\' characters, since all braces that aren't nested are supposed to be hidden in that way. For example, consider the process of skipping the first comment below, where the string containing the right brace has been typed as `'\.\}'` in the `AWEB` file.

```
⟨Procedures in input_phase 119⟩ +≡
  procedure skip_comment is   { skips to next unmatched '}' }
    bal : eight_bits ← 0;   { excess of left braces }
    c : ASCII_code;   { current character }
  begin
    loop
      if loc > limit then
        get_line;
        if input_has_ended then
          err_print("!␣Input␣ended␣in␣mid-comment");
          return;
        end if;
      end if;
      c ← buffer(loc); incr(loc);
      ⟨Do special things when c = "@", "\", "{", "}"; return at end 135⟩;
    end loop;
  end skip_comment;
```

135. ⟨Do special things when c = `"@"`, `"\"`, `"{"`, `"}"`; **return** at end 135⟩ ≡

```
  if c = [@] then
    c ← buffer(loc);
    if (c ≠ [␣]) ∧ (c ≠ tab_mark) ∧ (c ≠ [*]) ∧ (c ≠ [z]) ∧ (c ≠ [Z]) then
      incr(loc);
    else
      err_print("!␣Section␣ended␣in␣mid-comment");
      decr(loc); return;
    end if;
  elsif (c = [\]) ∧ (buffer(loc) ≠ [@]) then
    incr(loc);
  elsif c = [{] then
    incr(bal);
  elsif c = [}] then
    if bal = 0 then
      return;
    end if;
    decr(bal);
  end if
```

This code is used in section 134.

136. Inputting the next token. As stated above, ATANGLE's most interesting input procedure is the *get_next* routine that inputs the next token. However, the procedure isn't especially difficult.

In most cases the tokens output by *get_next* have the form used in replacement texts, except that two-byte tokens are not produced. An identifier that isn't one letter long is represented by the output '*identifier*', and in such a case the global variables *id_first* and *id_loc* will have been set to the appropriate values needed by the *id_lookup* procedure. Control codes produce the corresponding output of the *control_code* function above; and if that code is *module_name*, the value of *cur_module* will point to the *byte_start* entry for that module name.

Another global variable, *scanning_based_number*, is TRUE during the time that letters should be treated as if they were digits; *scanning_character* is set TRUE if the input 'x', where x is a character, or the ASCII code controlcode '@' is encoutered.

⟨ Types and objects visible from *data_structures* 10 ⟩ +≡
cur_module : *name_pointer*; { name of module just scanned }
scanning_character : BOOLEAN ← FALSE; { are we scanning a character literal? }

137. At the top level, *get_next* is a multi-way switch based on the next character in the input buffer. A *new_module* code is inserted at the very end of the input file.

```
⟨ Procedures in input_phase 119 ⟩ +≡
  function get_next return eight_bits is    { produces the next input token }
    c : eight_bits;   { the current character }
    d : eight_bits;   { the next character }
    k : long_name_index;   { index into mod_text }
  begin
  ⟪restart⟫ if loc > limit then
      get_line;
      if input_has_ended then
        return new_module;
      elsif scanning_based_number then
        err_print("!␣Improper␣based␣number,␣#␣sign␣missing");
        scanning_based_number ← FALSE;
      end if;
    end if;
    c ← buffer(loc); incr(loc);
    if scanning_based_number then
      if c = [#] then
        scanning_based_number ← FALSE;
      end if;
      return c;
    elsif scanning_character then
      if c = [@] then
        if buffer(loc) = [@] then
          incr(loc);
        else
          err_print("!␣You␣should␣double␣@␣signs␣in␣character␣literals");
        end if;
      end if;
      incr(loc); scanning_character ← FALSE; return c;
    end if;
    case c is
      when [#] ⇒
        if (loc > 1) and then (buffer(loc − 2) ∈ [0] .. [9]) then
          scanning_based_number ← TRUE;
        end if;
        return [#];
      when [A] .. [Z] | [a] .. [z] ⇒ ⟨ Get an identifier 139 ⟩;
      when [@] ⇒ ⟨ Get control code and possible module name 140 ⟩;
      ⟨ Compress two-symbol combinations like ':=' 138 ⟩
      when [␣] | tab_mark ⇒ goto restart;   { ignore spaces and tabs }
      when [{] ⇒ skip_comment; goto restart;
      when [´] ⇒ ⟨ If this is the beginning of an character token then set scanning_character to TRUE and
              return character_tok; otherwise do nothing 146 ⟩;
      when others ⇒ null;
    end case;
    return c;
  end get_next;
```

138.

```
define compress(#) ≡
        if loc ≤ limit then
          c ← #; incr(loc);
        end if
⟨Compress two-symbol combinations like ':=' 138⟩ ≡
when [.] ⇒
  if buffer(loc) = [.] then
    compress(double_dot);
  end if;
when [:] ⇒
  if buffer(loc) = [=] then
    compress(left_arrow);
  end if;
when [=] ⇒
  if buffer(loc) = [=] then
    compress(equivalence_sign);
  elsif buffer(loc) = [>] then
    compress(right_arrow);
  end if;
when [>] ⇒
  if buffer(loc) = [=] then
    compress(greater_or_equal);
  elsif buffer(loc) = [>] then
    compress(right_label_bracket);
  end if;
when [<] ⇒
  if buffer(loc) = [=] then
    compress(less_or_equal);
  elsif buffer(loc) = [<] then
    compress(left_label_bracket);
  elsif buffer(loc) = [>] then
    compress(box_sign);
  end if;
when [/] ⇒
  if buffer(loc) = [=] then
    compress(not_equal);
  end if;
when [*] ⇒
  if buffer(loc) = [*] then
    compress(double_star);
  end if;
```

This code is used in section 137.

139. We have to look at the preceding character to make sure this isn't part of a real constant, before trying to find an identifier starting with '`e`' or '`E`'.

```
⟨ Get an identifier 139 ⟩ ≡
  if (((c = [e]) ∨ (c = [E])) ∧ (loc > 1)) and then ((buffer(loc − 2) ∈ [0] .. [9]) ∨ buffer(loc − 2) = [#]) then
    c ← 0;
  end if;
  if c ≠ 0 then
    decr(loc); id_first ← loc;
    repeat
      incr(loc); d ← buffer(loc);
    until (((d < [0]) ∨ ((d > [9]) ∧ (d < [A])) ∨ ((d > [Z]) ∧ (d < [a])) ∨ (d > [z])) ∧ (d ≠ [_]));
    if loc > id_first + 1 then
      id_loc ← loc; return identifier;
    end if;
  else
    return [E];   { exponent of a real constant }
  end if
```

This code is used in section 137.

140. After an `@` sign has been scanned, the next character tells us whether there is more work to do.

```
⟨ Get control code and possible module name 140 ⟩ ≡
  c ← control_code(buffer(loc)); incr(loc);
  if c = ignore then
    goto restart;
  elsif c = module_name then
    ⟨ Scan the module name and make cur_module point to it 141 ⟩;
  elsif c = control_text then
    repeat
      c ← skip_ahead;
    until (c ≠ [@]);
    if buffer(loc − 1) ≠ [>] then
      err_print("!␣Improper␣@␣within␣control␣text");
    end if;
    goto restart;
  elsif c = ascii_tok then
    if (buffer(loc + 1) = [´]) or else (buffer(loc .. loc + 2) = ([@], [@], [´])) then
      scanning_character ← TRUE;
    else
      err_print("!␣Improper␣ASCII-code␣sequence,␣´␣sign␣mising");
    end if;
  end if
```

This code is used in section 137.

141.

```
⟨ Scan the module name and make cur_module point to it 141 ⟩ ≡
  ⟨ Put module name into mod_text(1 .. k) 143 ⟩;
  if (k > 3) and then (mod_text(k − 2 .. k) = ([.], [.], [.])) then
    cur_module ← prefix_lookup(k − 3);
  else
    cur_module ← mod_lookup(k);
  end if
```

This code is used in section 140.

142. Module names are placed into the *mod_text* array with consecutive spaces, tabs, and carriage-returns replaced by single spaces. There will be no spaces at the beginning or the end. (We set *mod_text*(0) ← [␣] to facilitate this, since the *mod_lookup* routine uses *mod_text*(1) as the first character of the name.)

```
⟨ Set initial values 15 ⟩ +≡
  mod_text(0) ← [␣];
```

143.

```
⟨ Put module name into mod_text(1 .. k) 143 ⟩ ≡
  k ← 0;
  loop
    if loc > limit then
      get_line;
      if input_has_ended then
        err_print("!␣Input␣ended␣in␣section␣name");
        exit;
      end if;
    end if;
    d ← buffer(loc); ⟨ If end of name, exit 144 ⟩;
    incr(loc);
    if k < longest_name − 1 then
      incr(k);
    end if;
    if (d = [␣]) ∨ (d = tab_mark) then
      d ← [␣];
      if mod_text(k − 1) = [␣] then
        decr(k);
      end if;
    end if;
    mod_text(k) ← d;
  end loop;
  ⟨ Check for overlong name 145 ⟩;
  if (mod_text(k) = [␣]) ∧ (k > 0) then
    decr(k);
  end if
```

This code is used in section 141.

144.

```
⟨ If end of name, exit 144 ⟩ ≡
  if d = [@] then
    d ← buffer(loc + 1);
    if d = [>] then
      loc ← loc + 2; exit;
    elsif (d = [␣]) ∨ (d = tab_mark) ∨ (d = [*]) then
      err_print("!␣Section␣name␣didn´t␣end");
      exit;
    end if;
    incr(k); mod_text(k) ← [@]; incr(loc);  { now d = buffer(loc) again }
  end if
```

This code is used in section 143.

145. ⟨ Check for overlong name 145 ⟩ ≡

```
if k ≥ longest_name − 2 then
  print_nl("!␣Section␣name␣too␣long:␣");
  for j ∈ INTEGER range 1 .. 25 loop
    print(xchr(mod_text(j)));
  end loop;
  print("...");  mark_harmless;
end if
```

This code is used in section 143.

146. ⟨ If this is the beginning of an character token then set *scanning_character* to TRUE and **return** *character_tok*; otherwise do nothing 146 ⟩ ≡

```
if (buffer(loc + 1) = ['´']) or else (buffer(loc .. loc + 2) = (['@'], ['@'], ['´'])) then
  scanning_character ← TRUE; return character_tok;
end if
```

This code is used in section 137.

147. Scanning a macro definition. The rules for generating the replacement texts corresponding to simple macros, parametric macros, and ADA texts of a module are almost identical, so a single procedure is used for all three cases. The differences are that

a) The sign `#` denotes a parameter only when it appears outside of strings in a parametric macro; otherwise it stands for the ASCII character `#`.
b) Module names are not allowed in simple macros or parametric macros; in fact, the appearance of a module name terminates such macros and denotes the name of the current module.
c) The symbols `@d` and `@f` and `@a` are not allowed after module names, while they terminate macro definitions.

148. Therefore there is a procedure *scan_repl* whose parameter t specifies either *simple* or *parametric* or *module_name*. After *scan_repl* has acted, *cur_repl_text* will point to the replacement text just generated, and *next_control* will contain the control code that terminated the activity.

```
⟨ Types and objects visible from data_structures 10 ⟩ +≡
  cur_repl_text : text_pointer;  { replacement text formed by scan_repl }
  next_control : sixteen_bits;
```

149.

```
⟨ Procedures in input_phase 119 ⟩ +≡
  procedure scan_repl(t : eight_bits) is   { creates a replacement text }
    a : sixteen_bits;  { the current token }
    b : ASCII_code;  { a character from the buffer }
    bal : eight_bits ← 0;  { left parentheses minus right parentheses }
  begin
    loop
    ⟨⟨continue⟩⟩ a ← get_next;
      case a is
        when [(] ⇒ incr(bal);
        when [)] ⇒ if bal = 0 then err_print("!␣Extra␣)"); else decr(bal);
          end if;
        when ["] ⇒ ⟨ Copy a string from the buffer to tok_mem 152 ⟩;
        when character_tok | ascii_tok ⇒ app_repl(a); a ← get_next;
        when [#] ⇒ if t = parametric then a ← param;
          end if;
        ⟨ In cases that a is a non-ASCII token (identifier, module_name, etc.), either process it and change
            a to a byte that should be stored, or goto continue if a should be ignored, or exit when a
            signals the end of this replacement text 151 ⟩
        when others ⇒ null;
      end case;
      app_repl(a);  { store a in tok_mem }
    end loop;
    next_control ← a;
    ⟨ Make sure the parentheses balance 150 ⟩;
    if text_ptr > max_texts − zz then
      overflow("text");
    end if;
    cur_repl_text ← text_ptr; tok_start(text_ptr + zz) ← tok_ptr(z); incr(text_ptr);
    if z = zz − 1 then
      z ← 0;
    else
      incr(z);
    end if;
  end scan_repl;
```

150. ⟨ Make sure the parentheses balance 150 ⟩ ≡

```
if bal > 0 then
  if bal = 1 then
    err_print("!␣Missing␣)");
  else
    new_ln; print("!␣Missing␣"); print_int(bal, 1); print("␣)´s"); error;
  end if;
  while bal > 0 loop
    app_repl([)]); decr(bal);
  end loop;
end if
```

This code is used in section 149.

151. ⟨ In cases that a is a non-ASCII token (*identifier*, *module_name*, etc.), either process it and change a to a byte that should be stored, or **goto** *continue* if a should be ignored, or **exit when** a signals the end of this replacement text 151 ⟩ ≡

```
when identifier ⇒ a ← id_lookup(normal); app_repl((a/400_8) + 200_8); a ← a mod 400_8;
when module_name ⇒
  exit when t ≠ module_name;
  app_repl((cur_module/400_8) + 250_8); a ← cur_module mod 400_8;
when verbatim ⇒ ⟨ Copy verbatim string from the buffer to tok_mem 153 ⟩;
when output_tok ⇒ ⟨ Copy output file-name from the buffer to tok_mem 154 ⟩;
when definition | format | begin_ada ⇒
  exit when t ≠ module_name;
  err_print("!␣@" & xchr(buffer(loc − 1)) & "␣is␣ignored␣in␣ADA␣text");
  goto continue;
when new_module ⇒ exit;
```

This code is used in section 149.

152. ⟨ Copy a string from the buffer to *tok_mem* 152 ⟩ ≡

```
b ← ["];
loop
  app_repl(b);
  if b = [@] then
    if buffer(loc) = [@] then
      incr(loc);  { store only one @ }
    else
      err_print("!␣You␣should␣double␣@␣signs␣in␣strings");
    end if;
  end if;
  if loc = limit then
    err_print("!␣String␣didn´t␣end");
    buffer(loc .. loc + 1) ← (["], 0);
  end if;
  b ← buffer(loc); incr(loc);
  if b = ["] then
    exit when buffer(loc) ≠ ["];
    incr(loc); app_repl(["]);
  end if;
end loop  { now a holds the final ["] that will be stored }
```

This code is used in section 149.

153. ⟨ Copy verbatim string from the buffer to *tok_mem* 153 ⟩ ≡

```
app_repl(verbatim); buffer(limit + 1) ← [@];
while buffer(loc) ≠ [@] loop
  app_repl(buffer(loc)); incr(loc);
  if (loc < limit) and then (buffer(loc .. loc + 1) = ([@], [@])) then
    app_repl([@]); loc ← loc + 2;
  end if;
end loop;
if loc ≥ limit then
  err_print("!␣Verbatim␣string␣didn´t␣end");
  if loc ≥ buf_size − 1 then
    loc ← buf_size − 2;
  end if;
elsif buffer(loc + 1) ≠ [>] then
  err_print("!␣You␣should␣double␣@␣signs␣in␣verbatim␣strings");
end if;
loc ← loc + 2   { another verbatim byte will be stored, since a = verbatim }
```

This code is used in section 151.

154. ⟨ Copy output file-name from the buffer to *tok_mem* 154 ⟩ ≡

```
app_repl(output_tok); buffer(limit + 1) ← [@];
while buffer(loc) ≠ [@] loop
  app_repl(buffer(loc)); incr(loc);
  if (loc < limit) and then (buffer(loc .. loc + 1) = ([@], [@])) then
    app_repl([@]); loc ← loc + 2;
  end if;
end loop;
if loc ≥ limit then
  err_print("!␣Output␣file␣name␣didn´t␣end");
elsif buffer(loc + 1) ≠ [>] then
  err_print("!␣You␣should␣double␣@␣signs␣in␣output␣file-names");
end if;
loc ← loc + 2   { another output_tok byte will be stored, since a = output_tok }
```

This code is used in section 151.

155. The following procedure is used to define a simple or parametric macro, just after the '==' of its definition has been scanned.

⟨ Procedures in *input_phase* 119 ⟩ +≡

```
procedure define_macro(t : eight_bits) is
  p : name_pointer;   { the identifier being defined }
begin
  p ← id_lookup(t); scan_repl(t);
  equiv(p) ← cur_repl_text; text_link(cur_repl_text) ← 0;
end define_macro;
```

156. Scanning a module. The *scan_module* procedure starts when '@␣' or '@*' has been sensed in the input, and it proceeds until the end of that module. It uses *module_count* to keep track of the current module number; with luck, AWEAVE and ATANGLE will both assign the same numbers to modules.

```
⟨ Types and objects visible from data_structures 10 ⟩ +≡
  module_count : mod_range;  { the current module number }
```

157. The top level of *scan_module* is trivial.

```
⟨ Procedures in input_phase 119 ⟩ +≡
  procedure scan_module is
    p : name_pointer;  { module name for the current module }
  begin
    incr(module_count);
    ⟨ Scan the definition part of the current module 158 ⟩;
    ⟨ Scan the ADA part of the current module 160 ⟩;
  end scan_module;
```

158.

```
⟨ Scan the definition part of the current module 158 ⟩ ≡
  next_control ← 0;
  loop
  ⟪continue⟫ while next_control ≤ format loop
      next_control ← skip_ahead;
      if next_control = module_name then  { we want to scan the module name too }
        loc ← loc − 2; next_control ← get_next;
      end if;
    end loop;
    exit when next_control ≠ definition;
    next_control ← get_next;  { get identifier name }
    if next_control ≠ identifier then
      err_print("!␣Definition␣flushed,␣must␣start␣with␣" & "identifier␣of␣length␣>␣1");
      goto continue;
    end if;
    next_control ← get_next;  { get token after the identifier }
    if next_control = equivalence_sign then
      define_macro(simple); goto continue;
    else
      ⟨ If the next text is '(#) ≡ ', call define_macro and goto continue 159 ⟩;
    end if;
    err_print("!␣Definition␣flushed␣since␣it␣starts␣badly");
  end loop
```

This code is used in section 157.

159. ⟨ If the next text is '(#) ≡ ', call *define_macro* and **goto** *continue* 159 ⟩ ≡

```
if next_control = [(] then
  next_control ← get_next;
  if next_control = [#] then
    next_control ← get_next;
    if next_control = [)] then
      next_control ← get_next;
      if next_control = [=] then
        err_print("!␣Use␣==␣for␣macros");
        next_control ← equivalence_sign;
      end if;
      if next_control = equivalence_sign then
        define_macro(parametric); goto continue;
      end if;
    end if;
  end if;
end if
```

This code is used in section 158.

160. ⟨ Scan the ADA part of the current module 160 ⟩ ≡

```
case next_control is
  when begin_ada ⇒
    p ← 0;
  when module_name ⇒
    p ← cur_module;
    ⟨ Check that = or ≡ follows this module name, otherwise return 161 ⟩;
  when others ⇒
    return;
end case;
⟨ Insert the module number into tok_mem 162 ⟩;
scan_repl(module_name);  { now cur_repl_text points to the replacement text }
⟨ Update the data structure so that the replacement text is accessible 163 ⟩
```

This code is used in section 157.

161. ⟨ Check that = or ≡ follows this module name, otherwise **return** 161 ⟩ ≡

```
repeat
  next_control ← get_next;
until (next_control ≠ [+]);  { allow optional '+=' }
if (next_control ≠ [=]) ∧ (next_control ≠ equivalence_sign) then
  err_print("!␣ADA␣text␣flushed,␣=␣sign␣is␣missing");
  repeat
    next_control ← skip_ahead;
  until (next_control = new_module);
  return;
end if
```

This code is used in section 160.

162. ⟨ Insert the module number into *tok_mem* 162 ⟩ ≡

$store_two_bytes(150000_8 + module_count)$ { $150000_8 = 320_8 * 400_8$ }

This code is used in section 160.

163. ⟨ Update the data structure so that the replacement text is accessible 163 ⟩ ≡

```
if p = 0 then  { unnamed module }
  text_link(last_unnamed) ← cur_repl_text; last_unnamed ← cur_repl_text;
elsif equiv(p) = 0 then
  equiv(p) ← cur_repl_text;  { first module of this name }
else
  p ← equiv(p);
  while text_link(p) < module_flag loop
    p ← text_link(p);  { find end of list }
  end loop;
  text_link(p) ← cur_repl_text;
end if;
text_link(cur_repl_text) ← module_flag  { mark this replacement text as a nonmacro }
```

This code is used in section 160.

164. ⟨ Procedures in *input_phase* 119 ⟩ +≡

```
procedure phase_i is
begin
  ⟨ Initialize the input system 126 ⟩;
  print_ln(banner);  { print a "banner line" }
  ⟨ Phase I: Read all the user's text and compress it into tok_mem 167 ⟩;
end phase_i;
```

165. The main program. We have defined plenty of procedures, and it is time to put the last pieces of the puzzle in place. Here is where `ATANGLE` starts, and where it ends.

define *web_input_file_name* ≡ `"input.aweb"`
define *change_input_file_name* ≡ `"input.ach"`
define *ada_output_file_name* ≡ `"web_output.a"`

⟨ Types and objects visible from *data_structures* 10 ⟩ +≡
opened : BOOLEAN ← FALSE; { was previous *open_a_file* succesfull? }

166.*

```
output to file atangle.adb
with Ada.Command_Line;
use Ada.Command_Line;
with TEXT_IO, int_number_io;
with data_structures, input_output, input_phase, output_phase;
use data_structures, input_output, input_phase, output_phase;
procedure ATANGLE is
  ac : INTEGER ← 0;
begin
  print_ln("Trying␣to␣open␣input␣file."); ac ← Argument_Count;
  if ac < 1 then
    fatal_error("No␣file␣on␣command␣line");
  end if;
  open_a_file(web_file, TEXT_IO.IN_FILE, Argument(1), opened);
  if ¬opened then
    fatal_error("AWEB␣file␣not␣open");
  end if;
  opened ← FALSE;
  if ac ≥ 2 then
    open_a_file(change_file, TEXT_IO.IN_FILE, Argument(2), opened);
  end if;
  if ¬opened then
    new_ln; print_ln("!␣No␣change␣file.");
  end if;
  open_a_file(ada_file, TEXT_IO.OUT_FILE, ada_output_file_name, opened);
  if (¬opened) and then (¬create_ada_file(ada_output_file_name)) then
    fatal_error("ADA␣file␣not␣open");
  end if;
  fn_length ← TEXT_IO.NAME(ada_file)'LAST; file_name(1 .. fn_length) ← TEXT_IO.NAME(ada_file);
  f_line ← 1; phase_i;
  stat
  for zo ∈ z_range loop
    max_tok_ptr(zo) ← tok_ptr(zo);
  end loop;
tats
phase_ii; raise end_of_ATANGLE; exception
when end_of_ATANGLE ⇒
  stat
    ⟨ Print statistics about memory usage 168 ⟩;
  tats
{ here files should be closed if the operating system requires it }
  ⟨ Print the job history 169 ⟩;
end ATANGLE;
```

167. ⟨ Phase I: Read all the user's text and compress it into *tok_mem* 167 ⟩ ≡

```
phase_one ← TRUE; module_count ← 0;
repeat
  next_control ← skip_ahead;
until (next_control = new_module);
while ¬input_has_ended loop
  scan_module;
end loop;
⟨ Check that all changes have been read 130 ⟩;
phase_one ← FALSE
```

This code is used in section 164.

168. ⟨ Print statistics about memory usage 168 ⟩ ≡

```
print_nl("Memory␣usage␣statistics:"); new_ln; print_int(name_ptr, 1); print("␣names,␣");
print_int(text_ptr, 1); print_ln("␣replacement␣texts;"); print_int(byte_ptr(0), 1);
for wo ∈ 1 .. ww − 1 loop
  print("+"); print_int(byte_ptr(wo), 1);
end loop;
print("␣bytes,␣"); print_int(max_tok_ptr(0), 1);
for zo ∈ 1 .. zz − 1 loop
  print("+"); print_int(max_tok_ptr(zo), 1);
end loop;
print("␣tokens.")
```

This code is used in section 166*.

169. Some implementations may wish to pass the *history* value to the operating system so that it can be used to govern whether or not other programs are started. Here we simply report the history to the user.

⟨ Print the job *history* 169 ⟩ ≡

```
case history is
  when spotless ⇒ print_nl("(No␣errors␣were␣found.)");
  when harmless_message ⇒ print_nl("(Did␣you␣see␣the␣warning␣message␣above?)");
  when error_message ⇒ print_nl("(Pardon␣me,␣but␣I␣think␣I␣spotted␣something␣wrong.)");
  when fatal_message ⇒ print_nl("(That␣was␣a␣fatal␣error,␣my␣friend.)");
end case; { there are no other cases }
new_ln
```

This code is used in section 166*.

170.* System-dependent changes. This module should be replaced, if necessary, by changes to the program that are necessary to make `ATANGLE` work at a particular installation. It is usually best to design your change file so that all changes to previous modules preserve the module numbering; then everybody's version will be consistent with the printed program. More extensive changes, which introduce new modules, can be inserted here; then only the index itself will get a new module number.

output to file `int_number_io.ads`

```
with TEXT_IO;
package int_number_io is new TEXT_IO.INTEGER_IO(INTEGER);
```

171.* Index. Here is a cross-reference table for the ATANGLE processor. All modules in which an identifier is used are listed with that identifier, except that reserved words are indexed only when they appear in format definitions, and the appearances of identifiers in module names are not indexed. Underlined entries of subprograms and packages correspond to sections where this entity is specified, whereas entries in italic type correspond to the section where the entity's body is stated. For any other identifier underlined entries correspond to where the identifier was declared. Error messages and a few other things like "ASCII code" are indexed here too.

The following sections were changed by the change file: 1, 3, 4, 5, 6, 7, 166, 170, 171.

`@a is ignored in ADA text`: 151.
`@d is ignored in ADA text`: 151.
`@f is ignored in ADA text`: 151.
a: 84, 149.
ac: 166.*
Ada: 166.*
ADA attributes: 13–14.
`ADA file not open`: 166.*
`ADA text flushed...`: 161.
ada_file: 25, 26, 94, 108, 166.*
ada_file_name: 23, 26.
ada_output_file_name: 165, 166.*
`Ambiguous prefix`: 69.
and_sign: 16, 102.
app: 96, 98–99.
app_repl: 90, 149–154.
Argument: 166.*
Argument_Count: 166.*
ASCII code: 12, 72.
ASCII code constant: 72.
ASCII_code: 12, 14, 16, 27–28, 39, 49, 62, 132, 134, 149.
ascii_tok: 72, 90, 102, 132, 140, 149.
ATANGLE: *166.**
`AWEB file ended...`: 124.
`AWEB file not open`: 166.*
b: 84, 94, 149.
bal: 84, 90, 134, 135, 149, 150.
banner: 1,* 164.
begin_ada: 131, 132, 151, 160.
begin_comment: 72, 112, 132.
BOOLEAN: 23–24, 26, 28–30, 92, 109, 116, 119, 136, 165.
box_sign: 16, 102, 138.
brace_level: 79, 80, 95, 112.
break_ptr: 91, 92–95, 98–99, 113.
buf_size: 10, 27–29, 153.
buffer: 27, 28–29, 32–33, 49, 51–52, 54, 56, 59, 119, 121, 123–130, 133–135, 137–140, 143–144, 146, 151–154.
buffer_index: 27, 29, 32, 49, 51, 116, 118.
buffer_type: 27, 91, 97, 118.
byte_field: 76.
byte_index: 39, 40, 51, 65, 69, 84, 101.
byte_mem: 38, 39, 40, 45, 48, 51, 54, 59, 61, 65–67, 69, 84, 101, 104.
byte_memory: 39.
byte_ptr: 40, 59, 66, 87, 168.
byte_start: 38, 39, 40–41, 48–49, 54, 59, 61, 66–67, 76, 78, 87, 104, 136.
c: 29, 51, 65, 69, 132, 133, 134, 137.
`Can't output ASCII code n`: 101.
carriage_return: 16–17, 28.
`Change file ended...`: 122, 124, 129.
`Change file entry did not match`: 130.
change_buffer: 118, 119–120, 123–124, 128, 130.
change_changing: 117, 124, 126, 129.
change_file: 22, 33, 116, 118, 121–122, 124, 129, 166.*
change_input_file_name: 165.
change_limit: 118, 119–120, 123–124, 128, 130.
changing: 33, 116, 117–118, 120, 124, 126–127, 130.
CHARACTER: 13, 15.
character_tok: 72, 90, 102, 146, 149.
check_break: 94, 98.
check_change: *124*, 128.
chop_hash: 49, 58, 60.
chop_hash_index: 49, 51.
chopped_id: 49, 51, 56, 61.
CLOSE: 108.
Command_Line: 166.*
comment_output: 92, 93–94, 112.
compare: 63, 65, 69.
compress: 138.
confusion: 36, 86.
continue: 101, 121, 149, 151, 158–159.
control_code: 131, *132*, 133, 136, 140.
control_text: 131, 132, 140.
`Could not open file 'x'`: 108.
count: 69.
CREATE: 26.
create_ada_file: 23, *26*, 108, 166.*
cur_byte: 76, 80–82, 84, 87, 90.
cur_char: 101, 104–105, 108, 110–112.
cur_end: 76, 80–82, 84, 87.
cur_mod: 76, 80–81, 84.
cur_module: 136, 141, 151, 160.

cur_name: 76, 80–82.
cur_repl: 76, 77, 80–82.
cur_repl_text: 148, 149, 155, 160, 163.
cur_state: 76, 81–82.
cur_val: 83, 84, 86, 104, 112.
d: 137.
data_structures: *3*,* 4*,* 7*,* 166*.
`data_structures.adb`: 3*.
`data_structures.ads`: 3*.
declare: 8.
decr: 9, 29, 82, 88, 90, 96, 112, 135, 139, 143, 149–150.
default_name: 23, 24.
define_macro: *155*, 158–159.
definition: 131, 132, 151, 158.
`Definition flushed...`: 158.
double_dot: 72, 102, 138.
double_star: 16, 102, 138.
eight_bits: 38, 39, 50–51, 72, 79, 84, 92, 97–98, 101, 131–134, 137, 149, 155.
end: 8–9.
end_comment: 72, 112, 132.
end_field: 76.
end_of_ATANGLE: 35, *166*.*
END_OF_FILE: 29.
END_OF_LINE: 29.
equal: 63, 65–67.
equiv: 38, 39, 44–46, 49, 58, 60–61, 66, 81, 85, 87, 155, 163.
equivalence_sign: 16, 102, 138, 158–159, 161.
err_print: 31, 57, 65, 69, 94, 105–106, 108, 111–112, 117, 121–122, 124–125, 129–130, 134–135, 137, 140, 143–144, 149–154, 158–159, 161.
error: 29, 31, *32*, 35, 61, 85, 87, 95, 101, 125, 150.
error_message: 11, 169.
extension: 63, 67.
`Extra )`: 149.
`Extra @}`: 112.
f: 24, 29.
f_line: 34, 94, 107, 108, 166*.
FALSE: 24, 26, 28–30, 93, 109–110, 112, 117–119, 124, 126, 136–137, 165–167.
fatal_error: 35, 36–37, 108, 166*.
fatal_message: 11, 169.
FILE: 24, 26.
`File name should have length > 0`: 108.
FILE_MODE: 23–24.
file_name: 34, 107, 108, 166*.
FILE_TYPE: 13.
final_limit: 29.
FIRST: 13.
flush_buffer: *94*, 95, 113.
fn_length: 34, 107, 108, 166*.
force_line: 72, 101, 132.
form_feed: 16, 28.
format: 131, 132, 151, 158.
found: 53–54, 65.
frac: 97, 98–99, 101, 111.
`Fraction too long`: 111.
GET: 29.
get_fraction: 101, 110–111.
get_line: 116, *127*, 133–134, 137, 143.
get_next: 136, *137*, 149, 158–159, 161.
get_output: 83, *84*, 91, 100–102, 105–106, 108, 110–112.
greater: 63, 65, 67, 69.
greater_or_equal: 16, 102, 138.
h: 51.
harmless_message: 11, 169.
hash: 40, 49, 53.
hash_index: 49, 51.
hash_size: 10, 49, 52, 56.
hashing: *5*,* 7*.
`hashing.adb`: 5*.
`hashing.ads`: 5*.
history: 11, 169.
history_type: 11.
`Hmm... n of the preceding...`: 125.
i: 18, 51.
id_first: 49, 51–52, 54, 56, 59, 136, 139.
id_loc: 49, 51–52, 54, 56, 59, 136, 139.
id_lookup: 49, 50, *51*, 136, 151, 155.
ident: 97, 98–99, 102, 104, 110.
identifier: 83, 86, 104, 136, 139, 151, 158.
`Identifier conflict...`: 61.
ignore: 131, 132–133, 140.
ilk: 38, 39, 44–45, 49, 55, 57–59, 82, 86–87.
`Improper @ within control text`: 140.
`Improper ASCII-code sequence...`: 140.
`Improper based number...`: 137.
IN_FILE: 166*.
`Incompatible module names`: 65.
incr: 9, 29, 52, 54, 56, 59, 61, 66–67, 69, 81, 84, 87, 90, 94, 96, 104–106, 108, 110–112, 121–122, 124, 128–129, 133–135, 137–140, 143–144, 149, 152–154, 157.
`Input ended in mid-comment`: 134.
`Input ended in section name`: 143.
`Input line too long`: 29.
input_has_ended: 116, 124, 126, 128, 133–134, 137, 143, 167.
input_ln: 28, *29*, 121–122, 124, 128–129.
input_output: 3*, *4*,* 5*7*,* 166*.
`input_output.adb`: 4*.

input_output.ads: 4.*
input_phase: *7*,* 166.*
input_phase.adb: 7.*
input_phase.ads: 7.*
int_number_io: 3*4,* 6*7,* 20, 166,* *170.**
int_number_io.ads: 170.*
INTEGER: 12, 15, 27, 38–39, 49, 62, 76, 83, 91, 96, 101, 107, 116, 124, 145, 166,* 170.*
INTEGER_IO: 170.*
IS_OPEN: 24, 26, 29.
j: 34, 65, 69, 101, 145.
join: 72, 98, 101, 132.
jump_out: 31, 35.
k: 33, 48, 51, 65, 69, 84, 94, 96, 98, 101, 137.
kj: 66.
kl: 61.
l: 32, 51, 65, 69.
LAST: 13, 166.*
last_unnamed: 70, 163.
left_arrow: 16, 102, 138.
left_label_bracket: 16, 102, 138.
length: 40, 53.
less: 63, 65–67, 69.
less_or_equal: 16, 102, 138.
limit: 28–29, 33, 116, 119, 121–123, 125–130, 133–134, 137–138, 143, 152–154.
line: 33, 93–94, 116, 117, 121–122, 124, 126, 128–130.
line_feed: 16, 28.
line_length: 10, 91, 94, 97, 101, 105–106, 111, 113.
lines_dont_match: *119*, 124.
link: 38, 39, 40, 45, 49, 53, 70.
llink: 45, 65–66, 69.
loc: 29, 33, 116, 121, 125–127, 129–130, 133–135, 137–140, 143–144, 146, 151–154, 158.
Long line must be truncated: 94.
long_name_index: 62, 65, 69, 137.
longest_name: 10, 62, 143, 145.
loop: 9.
mark_error: 11, 32.
mark_fatal: 11, 35.
mark_harmless: 11, 100, 145.
max_bytes: 10, 39, 59, 66.
max_id_length: 10, 104.
max_names: 10, 39, 59, 66, 87.
max_texts: 10, 39, 70, 87, 149.
max_tok_ptr: 42, 88, 166,* 168.
max_toks: 10, 39, 74, 90.
misc: 92, 93, 97–99, 101–102, 110, 112–113.
Missing n): 150.
mitpo: 8.
mod_field: 76.
mod_lookup: 62–63, 64, *65*, 141–142.
mod_range: 76, 156.
mod_text: 62, 65–69, 137, 141–145.
MODE: 24, 26.
mode_of_file: 23, 24.
module_count: 132, 156, 157, 162, 167.
module_flag: 70, 82, 163.
module_name: 131, 132, 136, 140, 148, 151, 158, 160.
module_number: 83, 84, 112.
n: 124.
NAME: 24, 26, 166.*
Name does not match: 69.
NAME_ERROR: 24, 26.
name_field: 76.
name_index: 39, 40, 69.
name_info: 39, 45.
name_pointer: 40, 47–48, 50–51, 64–65, 68–69, 76, 81, 136, 155, 157.
name_ptr: 40, 48, 51, 53, 55, 57, 59, 66, 87–90, 168.
NEW_LINE: 20.
new_ln: 20, 33, 94–95, 101, 125, 150, 166,* 168–169.
new_module: 131, 132–133, 137, 151, 161, 167.
next_control: 148, 149, 158–161, 167.
No change file.: 166.*
No output was specified: 100.
No parameter given for macro: 87.
normal: 44, 49–51, 55, 57–59, 86, 151.
Not present: <section name>: 85.
not_equal: 16, 102, 138.
not_found: 61.
not_sign: 16, 102.
null_array: 40, 41–43, 49.
num_or_id: 92, 98–99, 102.
OPEN: 24.
open_a_file: 23, *24*, 165–166.*
opened: 165, 166.*
optim: 8.
or_sign: 16, 102.
other_line: 116, 117, 126, 130.
out_buf: 34, 91, 93–94, 96–97.
out_buf_size: 10, 91, 96.
out_buffer_index: 91, 94, 96.
out_contrib: 97, 98, 101–102, 104–106, 110–112.
OUT_FILE: 26, 166.*
out_ptr: 34, 91, 92–96, 98–99, 113.
out_state: 92, 93, 98–99, 101–102, 105, 112–113.
Output file name didn't end: 154.
output_phase: *6*,* 166.*
output_phase.adb: 6.*
output_phase.ads: 6.*
output_state: *76*.

output_tok: 72, 101, 108, 132, 151, 154.
overflow: 37, 59, 66, 74, 81, 87, 90, 149.
p: 48, 51, 65, 69, 81, 155, 157.
PACK: 39.
param: 72, 84, 90, 149.
parametric: 44, 50, 82, 86, 148–149, 159.
phase_i: 7,* *164*, 166.*
phase_ii: 6,* *114*, 166.*
phase_one: 30, 32, 167.
pop_level: *82*, 84, 87–88.
POS: 14.
prefix: 63, 67.
prefix_lookup: 68, *69*, 141.
prime_the_change_buffer: *120*, 126, 129.
print: 20, 33–34, 48, 61, 85, 90, 94–95, 101, 125, 132, 145, 150, 168.
print_id: 47, *48*, 85, 87.
print_int: 20, 33–34, 94–95, 101, 125, 132, 150, 168.
print_ln: 20, 33–34, 164, 166,* 168.
print_nl: 20, 29, 31, 35, 61, 85, 87, 100, 145, 168–169.
`Program ended at brace level n`: 95.
push_level: *81*, 85–86, 89.
PUT: 20, 94.
q: 51, 65, 69.
r: 69.
repeat: 9.
repl_field: 76.
restart: 84–87, 89, 99, 127, 137, 140.
reswitch: 101, 105, 110–112.
right_arrow: 16, 102, 138.
right_label_bracket: 16, 102, 138.
rlink: 45, 46, 65–66, 69.
s: 51.
scan_module: 156, *157*, 167.
scan_repl: 148, *149*, 155, 160.
scanning_based_number: 109, 110, 136–137.
scanning_character: 136, 137, 140, 146.
`Section ended in mid-comment`: 135.
`Section name didn't end`: 144.
`Section name too long`: 145.
semi_ptr: 91, 93–95, 98, 112.
send_out: 97, *98*, 100–102, 104–106, 110–113.
send_the_output: 100, *101*.
send_val: *96*, 100, 102, 112.
set_element_sign: 16, 102.
simple: 44, 50, 86–87, 148, 158.
sixteen_bits: 38, 39, 44, 49, 64–65, 68–70, 73–74, 76, 83–84, 98, 148–149.
skip_ahead: *133*, 140, 158, 161, 167.
skip_comment: *134*, 137.
SKIP_LINE: 29.
`Sorry, x capacity exceeded`: 37.
spotless: 11, 169.
stack: 76, 81–82.
stack_ptr: 76, 80–82, 84, 87, 101, 105–106, 108.
stack_size: 10, 76, 81.
STANDARD_OUTPUT: 20.
stat: 8.
STATUS_ERROR: 24, 26.
store_two_bytes: 73, *74*, 90, 162.
str: 97, 98, 102, 105–106, 110, 112–113.
STRING: 23–24, 26, 107.
`String didn't end`: 152.
`String too long`: 105.
success: 23, 24.
system dependencies: 1,* 13, 17, 20–21, 24, 26, 28, 33, 165, 169–170.*
t: 51, 98, 149, 155.
tab_mark: 16, 33, 132, 135, 137, 143–144.
tats: 8.
temp_line: 116, 117.
text_char: 13, 14–15, 18, 20, 29.
text_file: 13, 22–25, 28–29.
text_index: 39, 42.
TEXT_IO: 3*7,* 13, 20, 23, *24*, *26*, 29, 94, 108, 166,* 170.*
text_link: 38, 39, 42, 70–71, 80, 82, 87, 100, 155, 163.
text_pointer: 42, 70, 76, 148.
text_ptr: 42, 78, 87–88, 149, 168.
`This can't happen`: 36.
`This identifier has already...`: 57.
`This identifier was defined...`: 57.
tok_mem: 38, 39, 42, 70, 74, 76–78, 84, 87, 90, 149.
tok_ptr: 42, 74, 78, 87–88, 90, 149, 166.*
tok_start: 38, 39, 42–43, 70, 76, 80–82, 87–88, 149.
token_index: 39, 42.
token_info: 39.
token_memory: 39.
TRUE: 24, 26, 28–30, 112, 116–117, 119, 124, 126, 128, 130, 136–137, 140, 146, 167.
u: 96.
unambig_length: 10, 44, 49, 56, 61.
unbreakable: 92, 99, 101, 105, 112.
until: 9.
update_terminal: 21, 32, 94, 100, 132.
uppercase: 56, 61, 102, 110–111.
`Use == for macros`: 159.
USE_ERROR: 24, 26.
v: 96, 98.
VAL: 14.
verbatim: 72, 101, 106, 132, 151, 153.

`Verbatim string didn't end`: 153.
`Verbatim string too long`: 106.
w: 48, 51, 65, 69, 84, 101.
w_range: 39, 40, 48, 51, 65, 69, 84, 101.
web_file: 22, 33, 116, 118, 124, 128, 130, 166*.
web_input_file_name: 165.
`Where is the match...`: 121, 125, 129.
wo: 168.
ww: 10, 39, 40–41, 48, 54, 59, 61, 66–67, 87, 104, 168.
x: 74.
xchr: 14, 15, 17–18, 33–34, 48, 61, 94, 108, 145, 151.
xord: 14, 18, 29.
`You should double @ signs`: 137, 152–154.
z: 42.
z_range: 39, 42, 77, 166*.
zo: 77, 80–82, 84, 87, 90, 166*, 168.
zz: 10, 39, 42–43, 77, 80–82, 87–88, 149, 168.

⟨ Cases involving `@{` and `@}` 112 ⟩ Used in section 101.
⟨ Cases like `/=` and `:=` 102 ⟩ Used in section 101.
⟨ Cases related to constants, possibly leading to *get_fraction* or *reswitch* 110 ⟩ Used in section 101.
⟨ Cases related to identifiers 104 ⟩ Used in section 101.
⟨ Check for ambiguity and update secondary hash 60 ⟩ Used in section 59.
⟨ Check for overlong name 145 ⟩ Used in section 143.
⟨ Check if q conflicts with p 61 ⟩ Used in section 60.
⟨ Check that all changes have been read 130 ⟩ Used in section 167.
⟨ Check that = or ≡ follows this module name, otherwise **return** 161 ⟩ Used in section 160.
⟨ Compare name p with current identifier, **goto** *found* if equal 54 ⟩ Used in section 53.
⟨ Compress two-symbol combinations like '`:=`' 138 ⟩ Used in section 137.
⟨ Compute the hash code h 52 ⟩ Used in section 51.
⟨ Compute the name location p 53 ⟩ Used in section 51.
⟨ Compute the secondary hash code h and put the first characters into the auxiliary array *chopped_id* 56 ⟩ Used in section 55.
⟨ Copy a string from the buffer to *tok_mem* 152 ⟩ Used in section 149.
⟨ Copy output file-name from the buffer to *tok_mem* 154 ⟩ Used in section 151.
⟨ Copy the parameter into *tok_mem* 90 ⟩ Used in section 87.
⟨ Copy verbatim string from the buffer to *tok_mem* 153 ⟩ Used in section 151.
⟨ Do special things when $c =$ `"@"`, `"\"`, `"{"`, `"}"`; **return** at end 135 ⟩ Used in section 134.
⟨ Empty the last line from the buffer 95 ⟩ Used in section 100.
⟨ Enter a new identifier into the table at position p 59 ⟩ Used in section 55.
⟨ Enter a new module name into the tree 66 ⟩ Used in section 65.
⟨ Expand macro a and **return**, or **goto** *restart* if no output found 86 ⟩ Used in section 84.
⟨ Expand module $a - 24000_8$, **goto** *restart* 85 ⟩ Used in section 84.
⟨ Force a line break 113 ⟩ Used in sections 101, 108, and 112(3).
⟨ Get an identifier 139 ⟩ Used in section 137.
⟨ Get control code and possible module name 140 ⟩ Used in section 137.
⟨ Get the buffer ready for appending the new information 99 ⟩ Used in section 98.
⟨ Get the *file_name* and redirect further output 108 ⟩ Used in section 101.
⟨ Give double-definition error and change p to type t 57 ⟩ Used in section 55.
⟨ If end of name, **exit** 144 ⟩ Used in section 143.
⟨ If the current line starts with `@y`, report any discrepancies and **return** 125 ⟩ Used in section 124.
⟨ If the next text is '(`#`) ≡ ', call *define_macro* and **goto** *continue* 159 ⟩ Used in section 158.
⟨ If this is the beginning of an character token then set *scanning_character* to TRUE and **return** *character_tok*; otherwise do nothing 146 ⟩ Used in section 137.
⟨ In cases that a is a non-ASCII token (*identifier*, *module_name*, etc.), either process it and change a to a byte that should be stored, or **goto** *continue* if a should be ignored, or **exit when** a signals the end of this replacement text 151 ⟩ Used in section 149.
⟨ Initialize the input system 126 ⟩ Used in section 164.
⟨ Initialize the output buffer 93 ⟩ Used in section 100.
⟨ Initialize the output stacks 80 ⟩ Used in section 100.
⟨ Insert the module number into *tok_mem* 162 ⟩ Used in section 160.
⟨ Make sure the parentheses balance 150 ⟩ Used in section 149.
⟨ Move *buffer* and *limit* to *change_buffer* and *change_limit* 123 ⟩ Used in sections 120 and 124.
⟨ Other printable characters 103 ⟩ Used in section 101.
⟨ Phase I: Read all the user's text and compress it into *tok_mem* 167 ⟩ Used in section 164.
⟨ Phase II: Output the contents of the compressed tables 100 ⟩ Used in section 114.
⟨ Print error location based on input buffer 33 ⟩ Used in section 32.
⟨ Print error location based on output buffer 34 ⟩ Used in section 32.
⟨ Print statistics about memory usage 168 ⟩ Used in section 166*.
⟨ Print the job *history* 169 ⟩ Used in section 166*.

⟨ Procedures in *data_structures* 48, 74 ⟩ Used in section 3*.
⟨ Procedures in *hashing* 51, 65, 69 ⟩ Used in section 5*.
⟨ Procedures in *input_output* 24, 26, 29, 32 ⟩ Used in section 4*.
⟨ Procedures in *input_phase* 119, 120, 124, 127, 132, 133, 134, 137, 149, 155, 157, 164 ⟩ Used in section 7*.
⟨ Procedures in *output_phase* 81, 82, 84, 94, 96, 98, 101, 114 ⟩ Used in section 6*.
⟨ Put a parameter on the parameter stack, or **goto** *restart* if error occurs 87 ⟩ Used in section 86.
⟨ Put module name into *mod_text*(1 . . *k*) 143 ⟩ Used in section 141.
⟨ Read from *change_file* and maybe turn off *changing* 129 ⟩ Used in section 127.
⟨ Read from *web_file* and maybe turn on *changing* 128 ⟩ Used in section 127.
⟨ Remove a parameter from the parameter stack 88 ⟩ Used in section 82.
⟨ Remove *p* from secondary hash table 58 ⟩ Used in section 57.
⟨ Scan the ADA part of the current module 160 ⟩ Used in section 157.
⟨ Scan the definition part of the current module 158 ⟩ Used in section 157.
⟨ Scan the module name and make *cur_module* point to it 141 ⟩ Used in section 140.
⟨ Send a string, **goto** *reswitch* 105 ⟩ Used in section 101.
⟨ Send verbatim string 106 ⟩ Used in section 101.
⟨ Set *c* to the result of comparing the given name to name *p* 67 ⟩ Used in sections 65 and 69.
⟨ Set initial values 15, 17, 18, 41, 43, 46, 71, 142 ⟩ Used in section 3*.
⟨ Skip over comment lines in the change file; **return** if end of file 121 ⟩ Used in section 120.
⟨ Skip to the next nonblank line; **return** if end of file 122 ⟩ Used in section 120.
⟨ Special code to finish real constants 111 ⟩ Used in section 101.
⟨ Specification of procedures visible from *data_structures* 47, 73 ⟩ Used in section 3*.
⟨ Specification of procedures visible from *hashing* 50, 64, 68 ⟩ Used in section 5*.
⟨ Specification of procedures visible from *input_output* 23, 28, 31 ⟩ Used in section 4*.
⟨ Start scanning current macro parameter, **goto** *restart* 89 ⟩ Used in section 84.
⟨ Types and objects visible from *data_structures* 10, 11, 12, 13, 14, 16, 22, 25, 27, 30, 35, 38, 39, 40, 42, 44, 45, 49, 62, 63, 70, 72, 76, 77, 79, 83, 91, 92, 97, 107, 109, 116, 118, 131, 136, 148, 156, 165 ⟩ Used in section 3*.
⟨ Update the data structure so that the replacement text is accessible 163 ⟩ Used in section 160.
⟨ Update the tables and check for possible errors 55 ⟩ Used in section 51.

The ATANGLE processor

(based on TANGLE Version 2.8)
[ADA-Version 1.0]

	Section	Page
Introduction	1	138
The character set	12	142
Input and output	19	145
Reporting errors to the user	30	148
Data structures	38	151
Searching for identifiers	49	154
Searching for module names	62	159
Tokens	70	162
Stacks for output	75	164
Producing the output	91	170
The big output switch	100	174
Introduction to the input phase	115	181
Inputting the next token	136	188
Scanning a macro definition	147	194
Scanning a module	156	197
The main program	165	200
System-dependent changes	170	203
Index	171	204